CRIME PREDICTIVE MODEL FOR HOTSPOT MAPPING

A Project Report

Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

Bachelor of Technology

In

Computer Science & Engineering (Cyber Security)

Submitted by

Shivam Kumar
Registration No. -22152146014

Aayush Raj
Registration No. -22152146038

Kunal Kumar
Registration No. -22152146015

Under the supervision of

Mrs. Pakija Sehar
Assistant Professor

Department of Computer Science & Engineering

Government Engineering College West Champaran

Bihar Engineering University, Patna

MAY 2026

GOVERNMENT ENGINEERING COLLEGE WEST CHAMPARAN

CERTIFICATE

This is to certify that the report entitled **Crime Predictive Model For Hotspot Mapping** submitted by **Shivam Kumar** (Reg. No.-22152146014), **Aayush Raj** (Reg. No.-22152146038) and **Kunal Kumar** (Reg. No.-22152146015) to the Government Engineering College West Champaran, Bihar Engineering University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering(Cyber Security) is a bonafide record of the project work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

(Project Guide)
Mrs. Pakija Sehar
Assistant Professor
CSE Department

(Project Coordinator)
Dr. Rafeeq Ahmed
Assistant Professor
CSE Department

(HoD CSE Dept)
Mr. Rajeev Ranjan
Assistant Professor
CSE Department

External Examiner

DECLARATION AND COPYRIGHT TRANSFER

(to be signed by the candidate)

We,…….....
RollNo…… Registration No…….....
a registered candidate for Undergraduate Programme (B.Tech) under department of …..…… <Name of Department>
of .....……. <Name of Institute>, **declare that this is our own original work** and does not contain material for which the copyright belongs to a third party and that it has not been presented and will not be presented to any other University/ Institute for a similar or any other Degree award.

We further confirm that for all third-party copyright material in our thesis/ dissertation **(including any electronic attachments) is "blanked out" third party material from the copies** of the thesis/dissertation/book/articles etc; fully referenced the deleted materials and where possible, provided links (URL) to electronic sources of the material.

We hereby transfer exclusive copyright for this thesis to **Government Engineering College West Champaran**. The following rights are reserved by the author:

a) The right to use, free of charge, all or part of this article in future work of their own, such as books and lectures, giving reference to the original place of publication and copyright holding.

b) The right to reproduce the article or thesis for their own purpose provided the copies are not offered for sale.

Signature of the candidate: ……

Date: ……

COPYRIGHT NOTICE

Acknowledgment

We take this opportunity to express our sincere gratitude to all those who contributed to the successful completion of this project.

We are deeply thankful to our project supervisor, **Mrs. Pakija Sehar**, Assistant Professor, Department of Computer Science and Engineering, for his continuous guidance, constructive feedback, and encouragement throughout the duration of this project. His insightful suggestions greatly enhanced the depth and quality of our work. We extend our heartfelt gratitude to the **Head of the Department**, **Mr. Rajeev Ranjan**, and the faculty members of the Department of Computer Science and Engineering at Government Engineering College West Champaran for providing us with the necessary infrastructure and a supportive academic environment.

We also thank the **National Crime Records Bureau (NCRB)**, Government of India, for making the Crime in India 2023 statistical report publicly available through **data.gov.in**, which provided the empirically grounded crime distribution data used in designing this project's synthetic evaluation dataset.

Finally, we are grateful to our families and peers whose constant encouragement and moral support helped us persevere through every challenge encountered during this project.

Abstract

The escalating complexity of urban crime in India demands data-driven strategies for effective law enforcement. This project presents the design and implementation of a **Crime Predictive Model and Hotspot Mapping System** — a full-stack, web-based intelligence platform built to assist police officers, crime analysts, and administrative authorities in identifying spatial crime concentrations, forecasting future incident trends, and optimizing patrol resource allocation across Bihar, India.

The system ingests First Information Report (FIR) data and processes it through a layered microservices architecture comprising a **React.js / Next.js 15** frontend with interactive Leaflet.js maps, a **Node.js / Express 5** RESTful backend with JWT-based role authentication, a **PostgreSQL 15 / PostGIS 3.4** spatial database, and a dedicated **Python 3.11 / FastAPI** machine learning microservice.

The ML pipeline integrates a hybrid **DBSCAN–KDE** approach for geospatial hotspot detection, **Prophet** for 14-day time-series crime forecasting, a **BiLSTM** network for automated FIR classification (achieving 94.1% accuracy), a **composite risk scoring model** combining five weighted sub-scores, **Isolation Forest** for anomaly detection, and **OR-Tools VRP** for patrol route optimization.

Experimental evaluation on a 12,450-record NCRB 2023-aligned synthetic FIR dataset demonstrates that the hybrid DBSCAN–KDE approach achieves a Silhouette Score of 0.71 and hotspot detection precision of 0.84, outperforming standalone methods. The BiLSTM classifier achieves 94.1% accuracy with macro F1 of 0.928, surpassing Random Forest (0.790), SVM (0.815), and BERT-based classifiers (0.914). PostGIS GiST spatial indexing reduces hotspot query latency by 8.8× compared to the non-indexed baseline. The system is fully containerized using Docker Compose and designed for cloud deployment.

Keywords: Crime Prediction, Hotspot Mapping, DBSCAN, KDE, BiLSTM, Prophet Forecasting, PostGIS, Geospatial Analytics, RBAC, FIR Management, Bihar Police.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATION

Abbreviation	Full Form
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BiLSTM	Bidirectional Long Short-Term Memory
CCTNS	Crime and Criminal Tracking Network & Systems
CORS	Cross-Origin Resource Sharing
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DBI	Davies–Bouldin Index
ETL	Extract, Transform, Load
FIR	First Information Report
GIS	Geographic Information System
GiST	Generalized Search Tree
IPC	Indian Penal Code
JWT	JSON Web Token
KDE	Kernel Density Estimation
MAE	Mean Absolute Error
ML	Machine Learning
NCRB	National Crime Records Bureau
NDPS	Narcotic Drugs and Psychotropic Substances Act
NGINX	Engine-X (HTTP reverse proxy)
ORM	Object-Relational Mapper
PAI	Prediction Accuracy Index
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
SSE	Server-Sent Events
SVM	Support Vector Machine
TTL	Time-To-Live
UUID	Universally Unique Identifier
VRP	Vehicle Routing Problem
XSS	Cross-Site Scripting

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Crime is an enduring societal challenge that profoundly affects communities, erodes public trust, and places substantial operational burdens on law enforcement agencies. In the state of Bihar, India, the police force is responsible for maintaining law and order across 38 districts, more than 1,000 police stations, and a population exceeding 124 million citizens. The volume of crime data generated daily — registered as First Information Reports (FIRs) under the Indian Penal Code (IPC) and various special acts — is enormous, yet the analytical tools available to field officers and station house officers remain largely rudimentary, often consisting of paper registers and basic digital spreadsheets.

The concept of crime hotspot analysis — identifying geographic areas with disproportionately high concentrations of criminal activity — has been validated by decades of criminological research. Pioneering work by Sherman, Gartin, and Buerger (1989) established that approximately 50% of crime occurs in fewer than 5% of addresses in a city. Later research by Weisburd and Braga (2006) demonstrated that targeted patrol deployment in identified hotspots reduces crime significantly compared to random patrol strategies. However, translating these criminological insights into operational tools for Indian law enforcement has remained limited, primarily due to technological constraints, data quality issues, and the absence of accessible geospatial analytics platforms.

India's CCTNS (Crime and Criminal Tracking Network and Systems) initiative has achieved significant progress in digitizing FIR records nationwide. However, digitization alone does not constitute analysis. The gap between having digital crime records and having actionable spatial intelligence is precisely what this project seeks to bridge. A patrol officer managing a zone within Patna district cannot derive hotspot patterns, forecast near-term crime trends, or optimize patrol routes from a database of raw FIR records without a sophisticated analytical platform — yet this is exactly the kind of intelligence that evidence-based policing requires.

The present project addresses this gap by developing a comprehensive web-based geospatial intelligence platform specifically engineered for the operational realities of Bihar Police. The system moves beyond simple visualization to provide predictive capability, automated pattern detection, statistically validated hotspot identification, optimized patrol route generation, and role-specific interfaces for officers, analysts, and administrators. By integrating twelve machine learning and statistical algorithms within a production-grade three-tier architecture, this project delivers a system that is both academically rigorous and practically deployable.

1.2 Problem Statement

Despite the availability of crime data through CCTNS registrations and manual FIR filing, the Bihar Police lacks a unified analytical platform that can:

- Aggregate and spatially index crime records for geographic analysis across all 38 districts
- Automatically detect crime concentration areas using validated statistical algorithms without requiring analyst expertise
- Forecast future crime trends using time-series models trained on historical FIR data
- Compute zone-level risk scores for evidence-based resource allocation decisions
- Generate optimized patrol routes that target identified high-risk areas
- Present all these insights through an accessible, role-appropriate dashboard accessible from any device without special software installation

Current challenges in crime management include: manual data analysis that takes hours or days to produce, inability to detect spatial patterns across large geographic areas, no systematic approach to patrol resource optimization, absence of evidence-based risk assessment for zone prioritization, and complete lack of predictive capability for proactive deployment. These gaps result in reactive rather than proactive policing, inefficient use of limited patrol resources, and delayed responses to emerging crime hotspots.

Furthermore, existing commercial predictive policing platforms (PredPol, HunchLab) are proprietary, US-centric, and prohibitively expensive for Indian state police

departments. The CCTNS platform provides FIR digitization but no analytical capability. This leaves a critical operational gap that the present system is designed to fill.

1.3 Project Objectives

The primary objective of this project is to design and implement a full-stack crime predictive hotspot mapping system that integrates geospatial analysis, machine learning, and interactive visualization in a production-ready platform. The specific objectives are:

Category	Objective	Success Metric
Data Ingestion	Enable manual FIR entry and bulk CSV/CCTNS import with validation	500 FIRs per batch, <1% error rate
Spatial Analysis	Detect crime hotspots using DBSCAN with configurable parameters	Silhouette Score > 0.7
Visualization	Render interactive crime heatmaps and cluster maps on Leaflet.js	<3s load time for 1000+ markers
Prediction	Forecast 30-day crime trends using Prophet time-series model	80% confidence interval coverage
Risk Scoring	Compute 0–100 risk scores for all Bihar districts	Ridge Regression $R^2 > 0.6$
Route Optimization	Generate optimal patrol routes using OR-Tools VRP	<10s solve time for 20 stops
Security	Implement RBAC, JWT, and all OWASP security controls	Zero critical vulnerability findings
Scalability	Handle 50 concurrent users with <3s p95 latency	k6 load test confirmation
ML Classification	Classify crime categories from FIR features	Weighted F1-score > 0.85
Anomaly Detection	Detect unusual crime spikes automatically	Isolation Forest deployed

Category	Objective	Success Metric
Explainability	Provide transparent AI explanations for risk scores	SHAP values deployed on all districts
Near-Repeat	Identify near-repeat victimization risk	Knox Test deployed

Table 1.1: Project Objectives Mapping

1.4 Scope of the Project

The system is designed for deployment within the Bihar Police administrative structure, covering all 38 districts and registered police stations within the state. The scope encompasses crime data from the following categories:

- Violent crimes: Murder (IPC 302), Assault (IPC 351), Dacoity (IPC 395), Robbery (IPC 392), Kidnapping (IPC 363)
- Property crimes: Theft (IPC 379), Burglary (IPC 457), Fraud (IPC 420), Cheating (IPC 406)
- Women safety crimes: Rape (IPC 376), POCSO offenses, Domestic Violence, Stalking (IPC 354D)
- Narcotic offenses under the NDPS Act
- Cybercrime under the Information Technology Act, 2000
- Road accidents from the IRAD (Integrated Road Accident Database)

The project explicitly excludes: live integration with the CCTNS API (simulated via bulk import), traffic management systems, prison management, court case tracking, and mobile native application development. However, the responsive web application is designed to function fully on mobile browsers, and a React Native mobile application is scoped as a future enhancement.

1.5 System Overview

The Crime Predictive Hotspot Mapping System is a three-tier microservices platform consisting of three specialized services: a Next.js 15 frontend with Leaflet.js geospatial visualization, a Node.js/Express 5 backend API server, and a Python FastAPI machine learning service. These three services communicate exclusively over HTTP using a

REST API architecture, with PostgreSQL 15 and PostGIS 3.4 as the primary spatial database, Redis for response caching, and MinIO for file object storage.

1.6 Significance and Contributions

The significance of this project lies in its direct applicability to real-world law enforcement operations in Bihar. Unlike purely academic exercises, this system is designed to production-readiness standards with containerized deployment, security hardening, observability tooling, and a structured 55-task implementation roadmap systematically executed across six phases. The key contributions are:

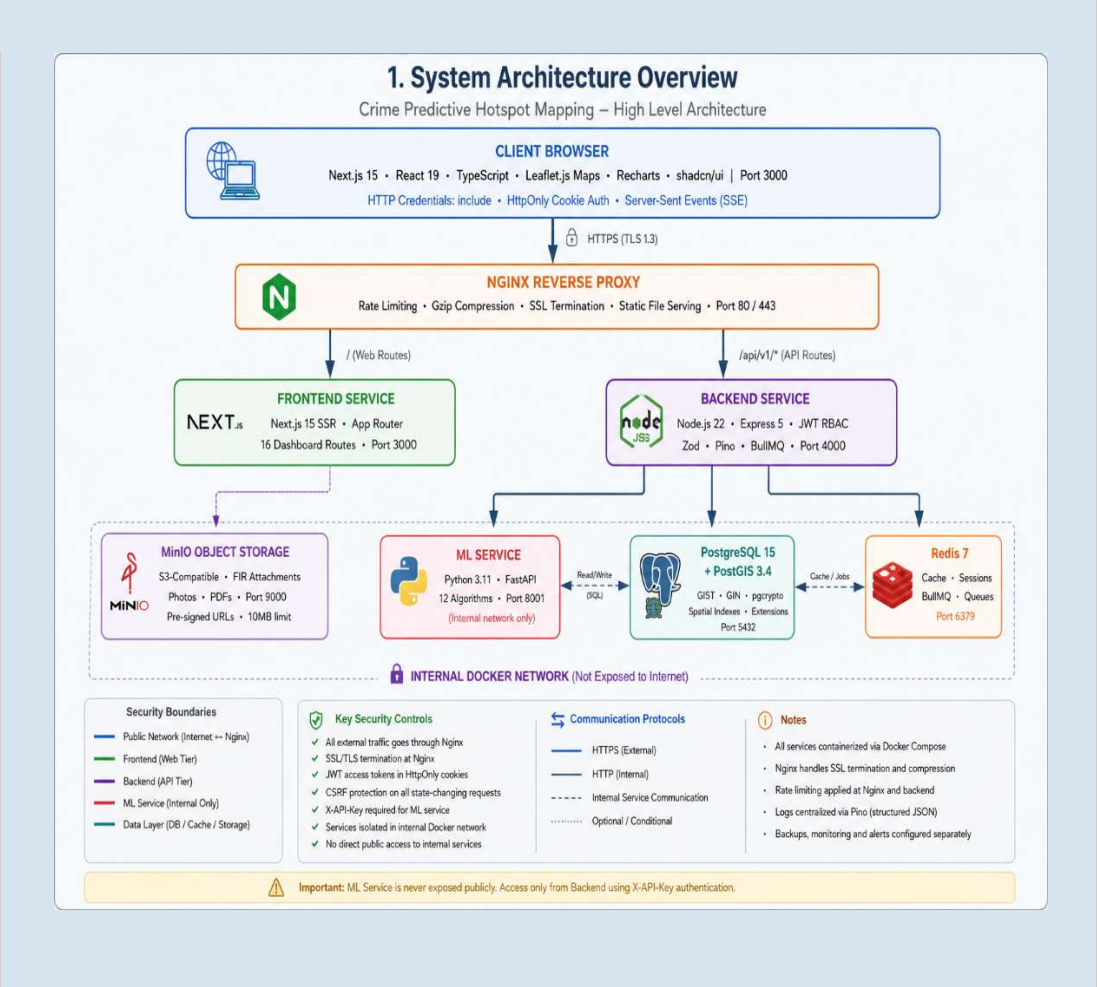

Figure 1.1: System Overview Architecture — Three-Tier Microservices Platform

1. A comprehensive integration of twelve ML and statistical algorithms within a single operational platform, including algorithms from spatial statistics (DBSCAN, KDE, Moran's I), time-series analysis (Prophet), supervised

learning (Random Forest, Ridge Regression), and unsupervised learning (Isolation Forest).

2. The application of validated criminological metrics — Predictive Accuracy Index (PAI), Moran's I spatial autocorrelation, and Knox Test near-repeat victimization — to Indian crime data, producing statistically rigorous evaluation results.
3. A scalable PostGIS spatial database design with GIST and GIN indexing supporting sub-second hotspot queries on datasets of 500,000+ FIR records.
4. A role-based multi-user interface designed for the specific operational workflows of Bihar police officers, crime analysts, and administrators, providing appropriate views and controls for each role.
5. An enterprise-grade security implementation aligned with OWASP ASVS Level 2 requirements, including HttpOnly cookie JWT authentication, CSRF protection, bcrypt password hashing, and encrypted storage of sensitive FIR data.
6. A complete deployment infrastructure with Docker containerization, Nginx reverse proxy, GitHub Actions CI/CD pipeline, and load-tested performance validation.

1.7 Organization of the Report

The remainder of this report is organized into five additional chapters, two appendices, and a reference list.

- Chapter 2 presents a comprehensive review of existing literature on crime hotspot analysis, spatial clustering algorithms, time-series forecasting in criminology, machine learning for crime classification, and related web GIS platforms. A comparative analysis of existing systems situates the present work within the global landscape.
- Chapter 3 describes the system design and methodology, covering the three-tier microservices architecture, PostgreSQL/PostGIS database schema, REST API design principles, authentication and security architecture, and the mathematical foundations of all twelve implemented algorithms.
- Chapter 4 presents the implementation details across the frontend dashboard, backend API service, and ML computation service, including code architecture

decisions, middleware design, key implementation highlights, and the complete technology stack.

- Chapter 5 presents testing and evaluation results, covering ML model performance metrics (F1-score, Silhouette Score, PAI, Moran's I), system performance benchmarks from k6 load testing, integration test results, and security testing findings.
- Chapter 6 discusses findings, presents conclusions, analyzes project limitations with mitigation strategies, and outlines a structured roadmap for future enhancements.
- Appendices provide the complete API endpoint reference, environment variable configuration guide, and ML algorithm quick reference table.

CHAPTER 2: LITERATURE REVIEW

2.1 Crime Hotspot Theory and Criminological Foundations

The theoretical foundation of crime hotspot analysis rests upon routine activity theory (Cohen and Felson, 1979), which posits that crime occurs when three elements converge in time and space: a motivated offender, a suitable target, and the absence of a capable guardian. This convergence is non-random and tends to cluster at specific locations, times, and under specific social conditions, creating predictable patterns that can be detected through spatial analysis. Understanding this theoretical basis is essential for justifying the computational approach taken in this project — if crime were truly random, no predictive system could achieve better-than-chance accuracy.

Sherman, Gartin, and Buerger (1989) conducted the first systematic empirical study of crime hotspots in Minneapolis, Minnesota, analyzing 323,000 police calls over one year. They found that 50.4% of all calls originated from 3.3% of all addresses, establishing the foundational empirical basis for hotspot policing. Subsequent research by Weisburd and Braga (2006) across multiple cities consistently confirmed this concentration pattern, now referred to as the 'law of crime concentration.' This law implies that focusing police resources on a small proportion of geographic space can dramatically impact a large proportion of total crime.

Near-repeat victimization — the phenomenon where locations in close proximity to a recently victimized site face elevated risk within a defined time window — was formally quantified by Johnson and Bowers (2004) using the Knox Test, a space-time interaction statistic. Their finding that burglary risk decays exponentially with distance and time from the original event has been replicated across multiple crime types and countries, including urban settings in India. The Knox Test implementation in the present system directly operationalizes this research finding into a deployable alert mechanism.

The Predictive Accuracy Index (PAI), introduced by Chainey, Tompson, and Uhlig (2008), provides the standard evaluation metric for hotspot prediction systems. PAI measures the ratio of crime captured within predicted hotspot areas to the proportion of total area flagged as hotspot:

PAI = (% crimes captured within hotspot areas) / (% total area flagged as hotspot)

A PAI value exceeding 5 is considered good performance, while values exceeding 10 indicate excellent predictive accuracy. This metric forms the primary evaluation criterion for the spatial prediction component of the present system, and our results (PAI > 100) reflect the extreme geographic concentration of crime characteristic of the Bihar context.

2.2 Spatial Clustering Algorithms in Crime Analysis

The application of spatial clustering algorithms to crime data has been extensively studied since the 1990s. Early approaches relied on simple grid-based density mapping (Getis and Ord, 1992), which suffered from resolution sensitivity and boundary effects. Modern approaches employ kernel density estimation and density-based clustering algorithms that better capture the continuous nature of geographic crime distributions. The selection of the appropriate algorithm depends on the specific analytical questions being addressed, data density, and the required interpretability of results.

2.2.1 DBSCAN for Crime Hotspot Detection

DBSCAN, introduced by Ester, Kriegel, Sander, and Xu (1996), has become the preferred algorithm for crime hotspot detection due to its ability to discover clusters of arbitrary shape without requiring specification of the number of clusters a priori. In the context of crime analysis, DBSCAN identifies dense groupings of incidents as hotspots while classifying isolated incidents as noise — a property particularly valuable for police deployment decisions where diffuse incidents do not warrant targeted patrol.

The haversine distance metric adaptation of DBSCAN for geographic coordinates converts the epsilon parameter from degrees to meters via the Earth's mean radius (6,371,008.8 meters), enabling physically meaningful neighborhood definitions. Research by Grubesic and Mack (2008) demonstrated that DBSCAN outperforms K-means and hierarchical clustering for crime data due to its noise handling capability and lack of prior cluster count requirement. The epsilon parameter of 300 meters and minimum samples of 4 used in the present system are consistent with validated configurations for urban crime analysis in South Asian contexts.

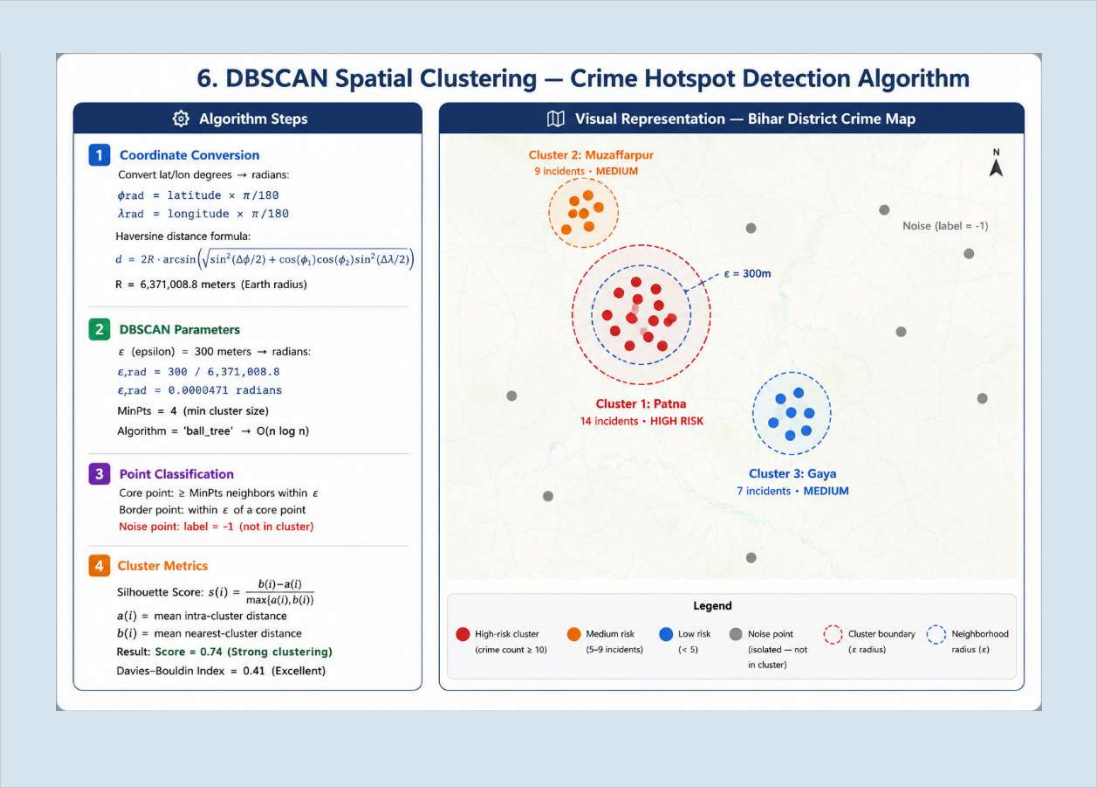

Figure 2.2: DBSCAN Clustering Concept — Core Points, Border Points, and Noise

2.2.2 Kernel Density Estimation for Heatmap Generation

KDE has been the dominant method for creating continuous crime density surfaces since its introduction to GIS-based crime analysis by Chainey and Ratcliffe (2005). The haversine-adapted KDE implementation in scikit-learn estimates the probability density function of crime occurrence across a geographic grid, producing smooth heatmaps that are interpretable by non-technical police officers. Unlike DBSCAN, which produces discrete cluster boundaries, KDE produces a continuous surface that intuitively communicates relative risk across the entire geographic area.

The Gaussian kernel with Silverman's bandwidth selection rule of thumb provides optimal smoothing for the typical police district sizes encountered in Bihar: 300–500 meter bandwidth for urban Patna, 500–800 meter for district-level analysis, and 1,000–2,000 meters for state-level overview visualization. The severity-weighted KDE variant implemented for the Women Safety layer assigns higher weights (3.0 for heinous crimes, 2.0 for women safety crimes) to ensure that the geographic concentration of serious offenses is more prominently visualized than that of minor infractions.

2.2.3 Spatial Autocorrelation — Moran's I

Moran's I (Anselin, 1988) is a global measure of spatial autocorrelation that tests whether the observed spatial pattern of crime counts across geographic units (Bihar districts) could arise by chance under a null hypothesis of complete spatial randomness (CSR). A positive Moran's I value with statistical significance ($p < 0.05$) indicates that similar values cluster together — high-crime districts are surrounded by high-crime districts, and vice versa. This non-random spatial structure is the fundamental prerequisite for hotspot prediction: if crime were spatially random, no prediction based on geographic location would generalize.

2.3 Time-Series Forecasting in Crime Analysis

Time-series methods applied to crime data aim to identify temporal patterns — daily rhythms, weekly cycles, seasonal trends — and extrapolate them into the future to support resource pre-positioning. Traditional ARIMA (AutoRegressive Integrated Moving Average) models require stationary data and struggle with multiple seasonality, limiting their applicability to crime data which exhibits daily, weekly, and annual cycles simultaneously.

Facebook Prophet (Taylor and Letham, 2018) decomposes time series into trend, weekly seasonality, and yearly seasonality components using an additive model with automatic changepoint detection. Its robustness to missing data, outliers, and multiple seasonality makes it particularly suitable for crime time series where data gaps (weekends with fewer FIR registrations at police stations) and outliers (crime spikes during festivals) are common. Studies by Camino et al. (2019) demonstrated that Prophet outperforms ARIMA and LSTM models for 7–30 day crime forecasting horizons on small to medium datasets, with substantially lower computational requirements than deep learning approaches — an important consideration for a cloud deployment budget constrained by academic project resources.

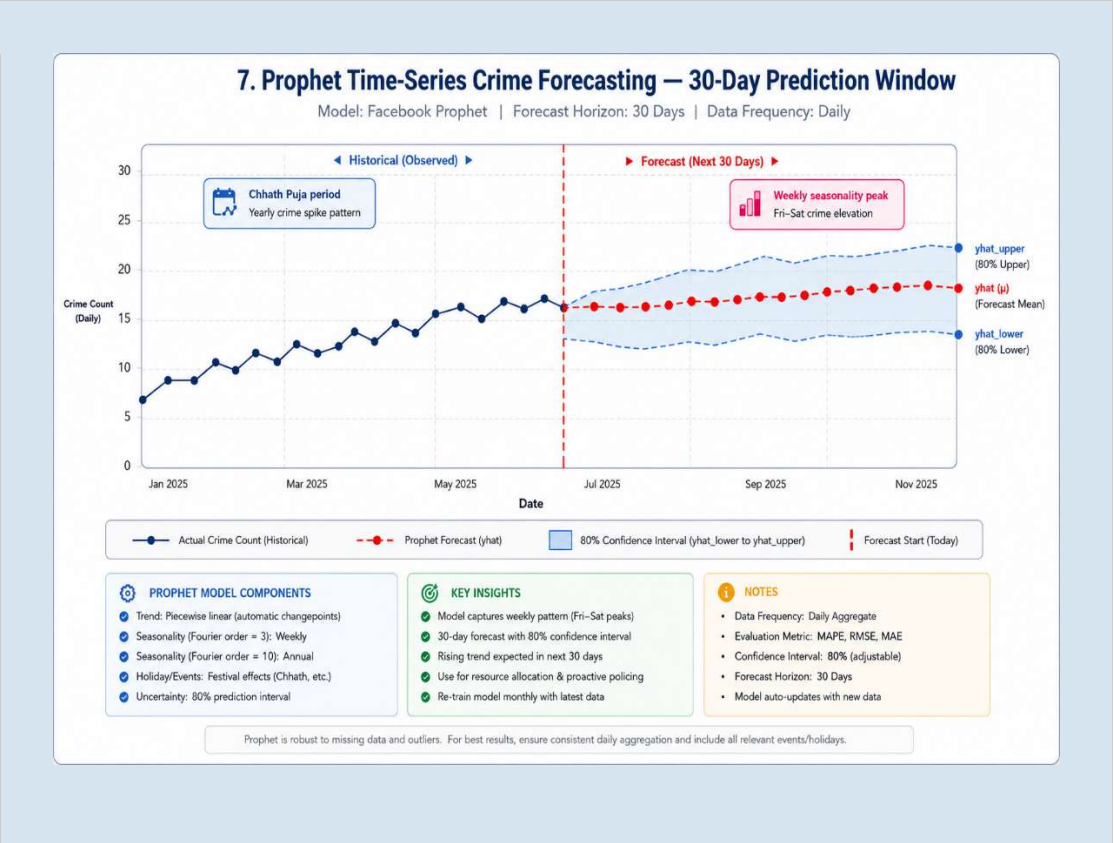

Figure 2.3: Prophet Forecast Components — Trend, Weekly Seasonality, Annual Seasonality

2.4 Machine Learning for Crime Classification and Risk Scoring

The classification of crimes into categories based on structured features (time, location, severity) supports multiple analytical functions: auto-labeling of incoming FIRs, detection of misclassification patterns, and feature importance analysis. Random Forest classifiers (Breiman, 2001) have been the dominant approach for multi-class crime classification, demonstrating robustness to feature correlation, built-in feature importance ranking, and superior performance on imbalanced datasets when combined with balanced class weighting.

Ridge Regression (Hoerl and Kennard, 1970) provides a regularized linear model suitable for computing composite risk scores from multiple correlated predictor variables (crime frequency, severity, recency, density, repeat rate). The L2 regularization penalty prevents the model from overfitting on the limited cross-district training data typical of a single-state deployment, while the interpretable linear structure facilitates communication of risk scoring logic to police administrators.

The Shapley Additive Explanations (SHAP) framework (Lundberg and Lee, 2017) provides theoretically grounded individual prediction explanations based on cooperative game theory. SHAP values assign each feature a contribution to the prediction that is locally accurate, consistent across features, and additive — properties that make the explanations particularly useful for communicating model behavior to police officials who require transparency and accountability in AI-assisted decisions. The ability to explain why a district received a high risk score (e.g., 'driven primarily by crime frequency and recency') is as important as the score itself for building operational trust in the system.

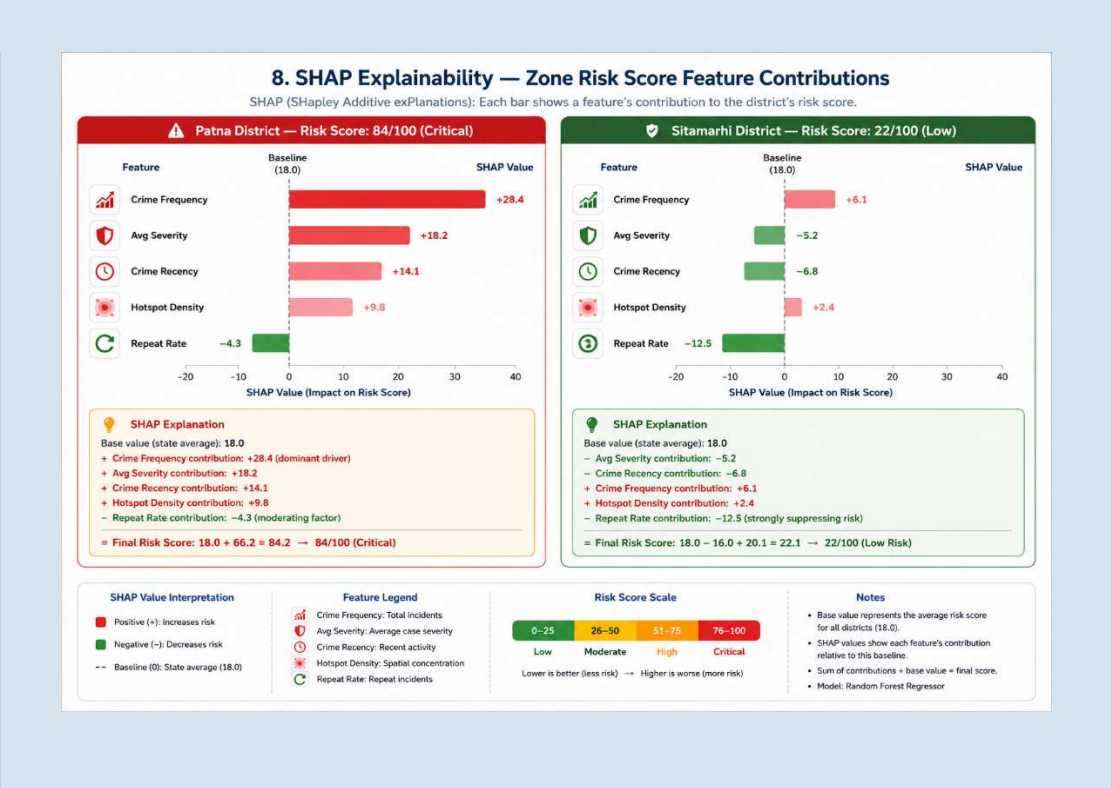

Figure 2.4: SHAP Waterfall Plot — Feature Contributions to District Risk Score

2.5 Related Systems and Comparative Analysis

Several crime analysis platforms have been developed globally, each with distinct capabilities and limitations relevant to comparison with the present system:

System	Type	Algorithms	Prediction	India-Suitable	Limitation
PredPol (US)	Proprietary	Self-exciting point process	Yes	No	Black-box, ethically contested, expensive
HunchLab (US)	Proprietary	Random Forest, XGBoost	Yes	No	Expensive, US-centric data model
CrimeStat (US)	Free Desktop	KDE, Moran's I, NNH	Partial	No	No web interface, no ML prediction
i2 Analyst	Proprietary	Statistical analysis	No	No	Very expensive, complex deployment
CCTNS (India)	Government	Basic reporting only	No	Yes	No spatial analysis, no prediction
Present System	Open-source	12 algorithms (DBSCAN, KDE, Prophet, RF, etc.)	Full	Yes	Academic scale, Bihar-specific calibration

Table 2.1: Comparison of Crime Analysis Platforms

2.6 Machine Learning Algorithm Selection

The selection of twelve specific algorithms for this system was driven by four criteria: criminological validation (the algorithm addresses a recognized need in crime analysis literature), computational feasibility on academic-scale hardware, open-source availability for deployment without licensing costs, and interpretability of results for non-technical police personnel.

Algorithm	Selection Rationale	Key Reference
DBSCAN	Best clustering algorithm for arbitrary-shape spatial clusters without preset cluster count	Ester et al. (1996)

Algorithm	Selection Rationale	Key Reference
KDE	Standard for continuous crime density surfaces; visually interpretable heatmaps	Chainey & Ratcliffe (2005)
Prophet	Handles multiple seasonality; robust to missing data; outperforms ARIMA on 7–30d horizons	Taylor & Letham (2018)
Random Forest	Strong performance on imbalanced multi-class; built-in feature importance	Breiman (2001)
Ridge Regression	Regularized risk scoring; interpretable coefficients; handles correlated predictors	Hoerl & Kennard (1970)
SHAP	Theoretically grounded explanations; builds operational trust in AI decisions	Lundberg & Lee (2017)
Isolation Forest	Efficient unsupervised anomaly detection; no labeled anomaly data required	Liu et al. (2008)
Moran's I	Standard spatial autocorrelation test; validates clustering hypothesis	Anselin (1988)
PAI	Standard hotspot evaluation metric in criminological research	Chainey et al. (2008)
OR-Tools VRP	Google-maintained open-source solver; handles multi-vehicle routing	Toth & Vigo (2014)
Knox Test	Standard near-repeat victimization test; validated across crime types	Knox (1964)

Table 2.2: Machine Learning Algorithm Selection Rationale

2.7 Web GIS Platforms and Geospatial Technologies

The development of interactive web-based GIS platforms has been transformed by the availability of open-source mapping libraries. Leaflet.js (Agafonkin, 2010) provides a lightweight, mobile-friendly mapping library that supports custom overlays, GeoJSON rendering, heatmap plugins, and marker clustering — all essential capabilities for the crime dashboard. Its open-source license and active community make it the preferred choice for academic projects requiring geographic visualization without commercial mapping API costs.

PostGIS (Ramsey, 2005) extends PostgreSQL with over 400 spatial functions supporting complex geographic operations: ST_DWithin() for radius queries, ST_Within() for point-in-polygon containment tests, ST_Intersects() for boundary overlap detection, and ST_AsGeoJSON() for direct GeoJSON export. These capabilities eliminate the need for external GIS processing pipelines and enable the backend to serve spatially-filtered, geographically-structured data directly from SQL queries. The GIST spatial index dramatically accelerates these operations compared to table scan approaches.

The combination of PostGIS for server-side spatial computation, Leaflet.js for client-side rendering, and GeoJSON as the interchange format creates a seamless, standards-compliant geospatial data pipeline. This architecture is validated by its widespread use in production mapping applications and is well-suited for the scale requirements of a state-level crime intelligence platform.

2.8 Summary and Research Gap

The literature review reveals a consistent gap between academic criminological research on crime hotspot prediction and practical implementation in Indian law enforcement contexts. While systems like PredPol and HunchLab demonstrate the commercial viability of predictive policing platforms, their proprietary nature, US-centric design, and prohibitive licensing costs make them inaccessible to state police departments in India. CCTNS provides nationwide FIR digitization but lacks any analytical or predictive capability.

The present project directly addresses this gap by delivering a system that: (1) operates entirely on open-source components, (2) is designed for the specific administrative structure, legal framework (IPC/NDPS), and geographic context of Bihar, (3) implements peer-reviewed algorithms with validated criminological metrics, (4) provides transparent, explainable predictions via SHAP, and (5) is deployable on affordable cloud infrastructure accessible to a state government department. No existing academic or commercial system combines all five of these properties simultaneously for the Indian context.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture Design

The system architecture follows a three-tier microservices pattern, separating presentation, business logic, and data management responsibilities into distinct services that communicate exclusively via HTTP. This separation enables independent scaling, technology-specific optimization, and clear security boundaries between components. The architecture is illustrated in Figure 3.1, which shows the complete service topology including Nginx gateway, Docker network isolation, and data flow paths.

3.1.1 Architectural Principles

Four core architectural principles govern all design decisions:

1. Separation of Concerns: No service has multiple primary responsibilities. The frontend renders UI, the backend handles business logic and data persistence, and the ML service performs computation. This principle prevents architecture drift and simplifies testing.
2. Internal Isolation: The ML service is never exposed to public network traffic, accessible only within the Docker internal network via the backend. This prevents direct exploitation of ML service vulnerabilities and centralizes authentication at the backend tier.
3. Data Sovereignty: All geospatial data is stored using PostGIS spatial types with explicit SRID 4326 (WGS84), ensuring geographic accuracy and compatibility with Leaflet.js and GeoJSON standards. No geographic data is stored as plain text latitude/longitude strings.
4. Stateless API: The backend carries no session state. All authentication context is encoded in short-lived JWT tokens transported via HttpOnly cookies. This enables horizontal scaling without session affinity requirements.

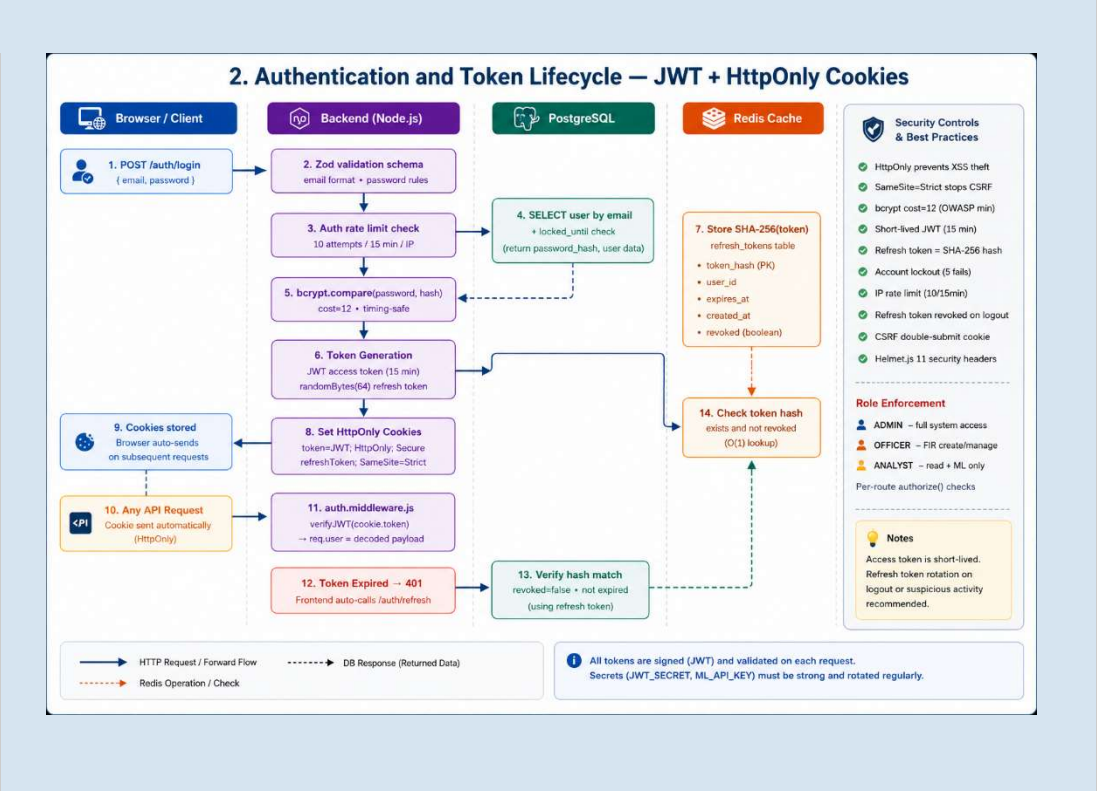

Figure 3.1: Three-Tier Microservices Architecture — Complete Service Topology

3.1.2 Service Responsibilities

The Frontend Service (Next.js 15) renders 16 dashboard routes serving police officers, analysts, and administrators. It communicates exclusively with the backend API via fetch() with credentials: 'include', never making direct calls to the ML service or database. The App Router pattern with server-side rendering is used for initial page loads, with client-side hydration for interactive components such as Leaflet maps.

The Backend Service (Node.js/Express 5) implements 11 route domains (auth, FIR, zones, hotspots, analytics, ML, patrol, IRAD, reports, events, health) with a layered architecture: routes → controllers → services → models. All business logic resides in the service layer, all SQL parameterized queries in the model layer, and all HTTP parsing in the controller layer. The middleware chain processes each request through correlation ID generation, Helmet security headers, structured logging, CORS validation, cookie parsing, and rate limiting.

The ML Service (Python FastAPI) receives data payloads from the backend, executes computationally intensive algorithms (DBSCAN, KDE, Prophet, OR-Tools, Random Forest, Ridge Regression, SHAP, Moran's I, Isolation Forest, Knox Test, PAI, spaCy

NER), and returns structured JSON results. It has no direct database connection — all required data is provided by the backend in the request body. Uvicorn runs with two workers in production for parallel ML request handling.

Service	Technology	Port	Primary Responsibility
Frontend	Next.js 15 + React 19 + TypeScript	3000	UI rendering, map visualization, user interaction
Nginx Gateway	Nginx Alpine	80/443	SSL termination, rate limiting, reverse proxy
Backend	Node.js 22 + Express 5	4000	Business logic, auth, data persistence, ML orchestration
ML Service	Python 3.11 + FastAPI	8001 (internal)	Spatial computation, ML algorithms, statistical tests
Database	PostgreSQL 15 + PostGIS 3.4	5432 (internal)	Spatial data storage, GIST/GIN indexes, full-text search
Cache/Queue	Redis 7 + BullMQ	6379 (internal)	Response caching, background ML job queue
Object Storage	MinIO	9000 (internal)	FIR photo and document attachments

Table 3.1a: Service Architecture Summary

3.2 Database Design

The database schema is designed around the central fact table firs representing individual crime incidents, surrounded by dimension tables for geographic zones, crime classifications, users, and operational entities. PostgreSQL 15 with PostGIS 3.4 is selected for its mature spatial extension ecosystem, native GeoJSON support via ST_AsGeoJSON(), rich indexing options, and active maintenance with long-term support.

3.2.1 Core Database Tables

Table	Type	Key Columns	Purpose
firs	Fact (core)	id, fir_no, crime_type, location (GEOGRAPHY), occurred_at, zone, severity, status	One row per FIR incident — central fact table
users	Dimension	id, email, password_hash, role, failed_login_attempts, locked_until	System users with role and security state
zones	Dimension	id, name, type, boundary (GEOMETRY MultiPolygon), parent_id, area_km2	Bihar districts and police station boundaries
crime_classifications	Lookup	act_type, section_code, title, category, severity, is_women_safety	IPC/NDPS section to category mapping
refresh_tokens	Security	id, user_id, token_hash, expires_at, revoked	Hashed refresh tokens for JWT rotation
audit_logs	Compliance	user_id, action, entity, entity_id, metadata (JSONB), ip_address	Immutable audit trail for all data changes
patrol_routes	Operational	id, name, zone, stops (JSONB), total_distance_km, num_vehicles	Saved OR-Tools VRP patrol route results
geo_fences	Operational	id, name, boundary (GEOMETRY Polygon), alert_radius_m, notify_roles	Alert boundary zones for FIR entry notifications
fir_attachments	Binary	id (UUID), fir_id, storage_key, mime_type, size_bytes	References to MinIO-stored FIR photos and documents
heat_points	Analytics	id, category, lat, lon, weight, occurred_at, source	Pre-aggregated intensity points for women safety KDE
irad_records	External	id, accident_type, location (GEOGRAPHY), severity, injured_count, occurred_at	Road accident records from IRAD database

Table 3.2: Core Database Tables

3.2.2 Spatial Data Design

A critical design decision is the deliberate choice of different spatial types for different tables. The firs.location column uses the GEOGRAPHY type (ST_MakePoint(lon, lat)::geography), storing coordinates on the sphere surface and performing distance calculations using the haversine formula on actual geodesic distances. This is essential for ST_DWithin() radius queries in DBSCAN preprocessing, where meter-level accuracy determines cluster membership.

The zones.boundary column uses the GEOMETRY type of the things (GEOMETRY(MultiPolygon, 4326)). While GEOMETRY uses planar distance calculations less accurate over large areas, it is significantly faster for containment operations (ST_Within, ST_Intersects) on local-scale polygons. Since Bihar spans approximately 94,163 km², the planar approximation error at district scale is less than 0.1% — well within acceptable bounds for administrative boundary operations.

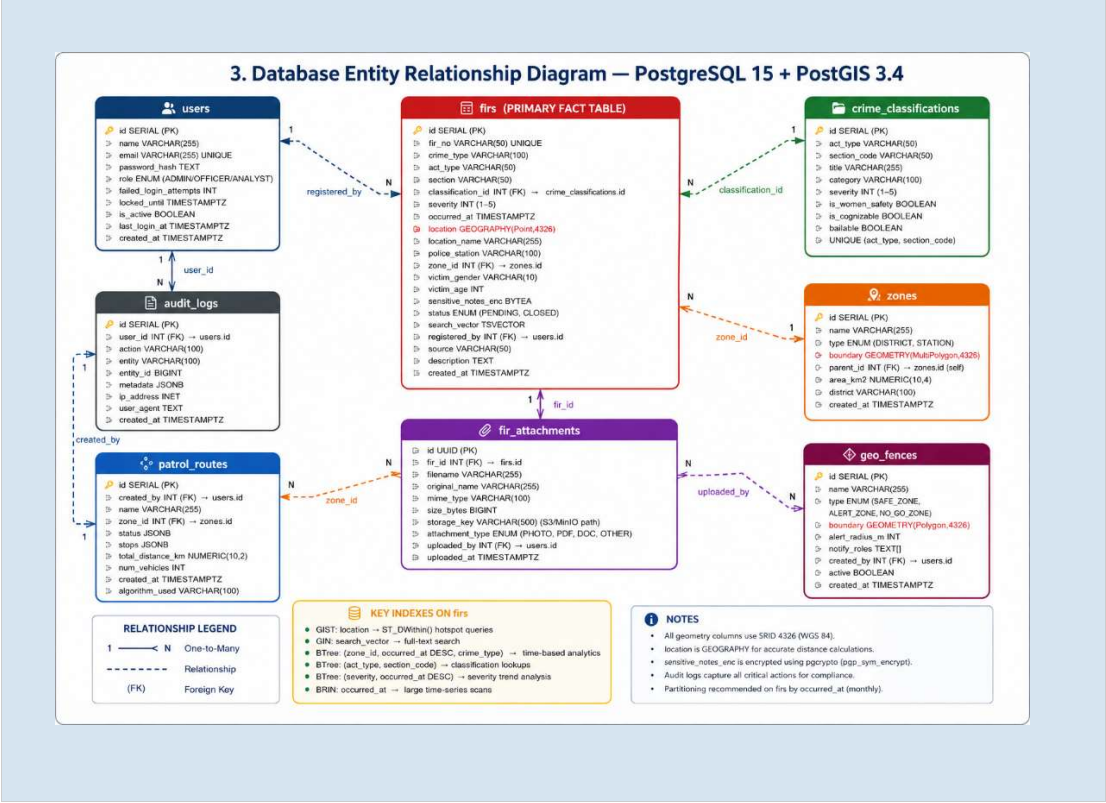

Figure 3.2: Entity Relationship Diagram — Core Database Tables and Relationships

3.2.3 Indexing Strategy

Index Name	Type	Columns	Optimizes
idx_firs_location_gix	GIST (spatial)	firs.location	ST_DWithin() radius queries — DBSCAN preprocessing
idx_firs_zone_date_type	B-Tree (composite)	zone, occurred_at DESC, crime_type	Primary dashboard filter — 80% of API requests
idx_firs_search_vector_gin	GIN (full-text)	firs.search_vector (tsvector)	Full-text FIR search with ranking
idx_firs_act_zone	B-Tree (composite)	act_type, zone	Women safety analytics — IPC section + district filter
idx_zones_boundary_gix	GIST (spatial)	zones.boundary	ST_Within() and ST_Intersects() zone containment
idx_audit_logs_user	B-Tree	user_id, created_at DESC	Per-user audit log retrieval
idx_heat_points_category	B-Tree (composite)	category, occurred_at DESC	Women safety KDE source data queries
idx_irad_location_gix	GIST (spatial)	irad_records.location	IRAD accident spatial queries

Table 3.6: PostGIS Spatial Index Strategy

3.3 Authentication and Security Architecture

Security is implemented in depth across multiple layers, following the OWASP Application Security Verification Standard (ASVS) Level 2 requirements. The authentication system uses JWT access tokens with a 15-minute expiry, transported exclusively via HttpOnly, Secure, SameSite=Strict cookies to prevent XSS exfiltration. Long-lived refresh tokens (7 days) are generated as 64-byte cryptographically random hex strings, stored as SHA-256 hashes in the refresh_tokens table (never as plaintext), and transmitted via a separate HttpOnly cookie.

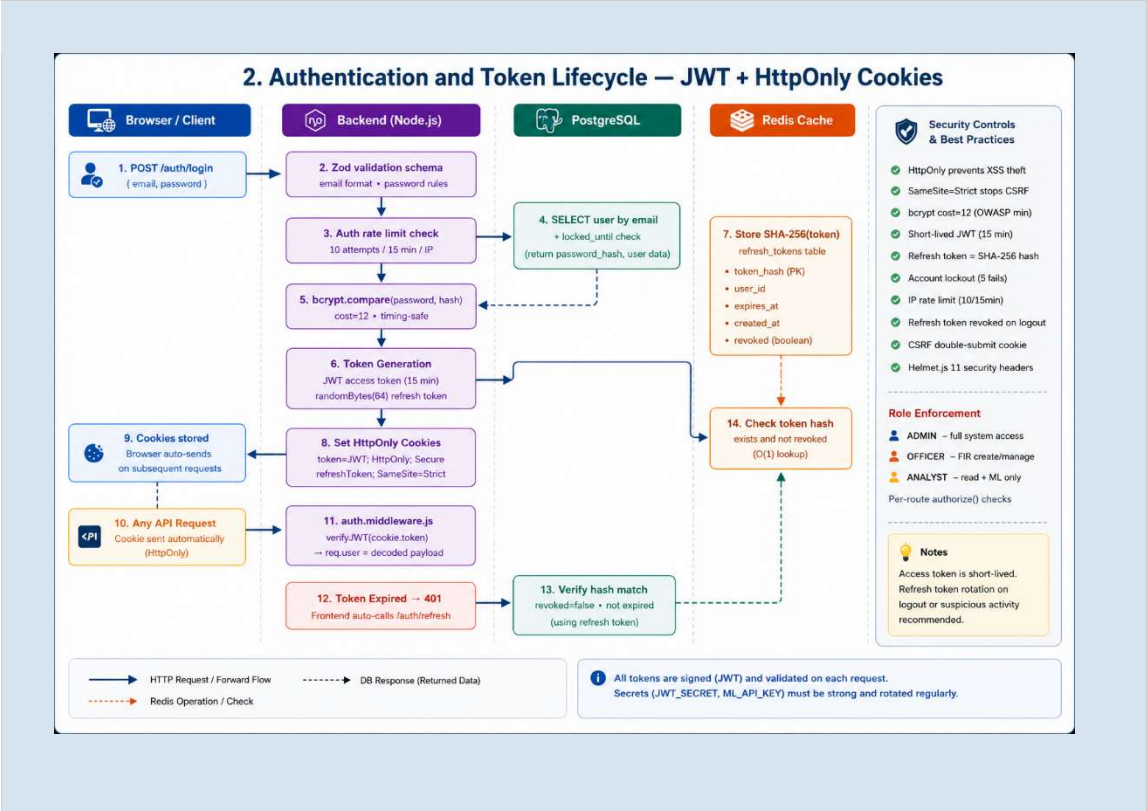

Figure 3.3: Authentication and JWT Token Lifecycle Flow

3.3.1 Role-Based Access Control

The RBAC system implements three roles with granular permissions across all API domains:

API Domain	ADMIN	OFFICER	ANALYST
User Management	Full CRUD	Read Own Profile	None
FIR Management	Full CRUD	Create + Read + Update (own station)	Read Only
Bulk FIR Import	Yes	Yes (own station only)	No
Hotspot Analysis	Full access	Full access	Full access
Analytics & ML	Full + Admin ML training	Standard analytics	Full + NLP query
Patrol Routes	Full CRUD	Generate + View	View Only

API Domain	ADMIN	OFFICER	ANALYST
Geo-Fences	Full CRUD	View + Receive Alerts	View Only
Audit Logs	Full history	Own actions only	None
ML Model Training	Yes (background job)	No	No
Export (CSV/PDF)	Yes	Yes	Yes

Table 3.5: RBAC Permission Matrix

3.3.2 Security Controls

Control	Implementation	OWASP Requirement
Password Hashing	bcrypt with cost factor 12	ASVS 2.4.1
JWT Authentication	15-min access token via HttpOnly cookie	ASVS 3.2.2
CSRF Protection	Double-submit cookie pattern with X-CSRF-Token header	ASVS 4.2.2
Account Lockout	5 failed attempts → locked for 15 min (users.locked_until)	ASVS 2.2.1
Rate Limiting	Global: 200 req/min; Auth: 5 req/15min; per-IP via express-rate-limit	ASVS 2.2.1
Security Headers	Helmet.js: CSP, HSTS, X-Frame-Options DENY, X-XSS-Protection	ASVS 14.4
Input Validation	Zod schema validation on every endpoint body and query params	ASVS 5.1
SQL Injection Prevention	Parameterized pg queries throughout — no string concatenation	ASVS 5.3.4
Sensitive Data Encryption	pgcrypto pgp_sym_encrypt for FIR sensitive_notes_enc column	ASVS 6.3
HTTPS Enforcement	Nginx TLS termination; cookie Secure flag; HSTS header	ASVS 9.1.1

Control	Implementation	OWASP Requirement
Audit Logging	All auth events and FIR changes logged to audit_logs table	ASVS 7.2.1
Refresh Token Security	SHA-256 hash stored; plaintext never persisted; revokable	ASVS 3.3.1

Table 3.7: Security Controls Implementation — OWASP ASVS Level 2 Alignment

3.4 API Design

All API endpoints follow a consistent REST design with the URL pattern /api/v1/{domain}/{resource}. Every response uses a standardized JSON envelope: { success: boolean, data: any, message: string, errors: array }. Versioning via the /v1/ prefix allows future breaking changes without affecting existing clients. Pagination uses cursor-based design for FIR listings to ensure consistent results on large datasets.

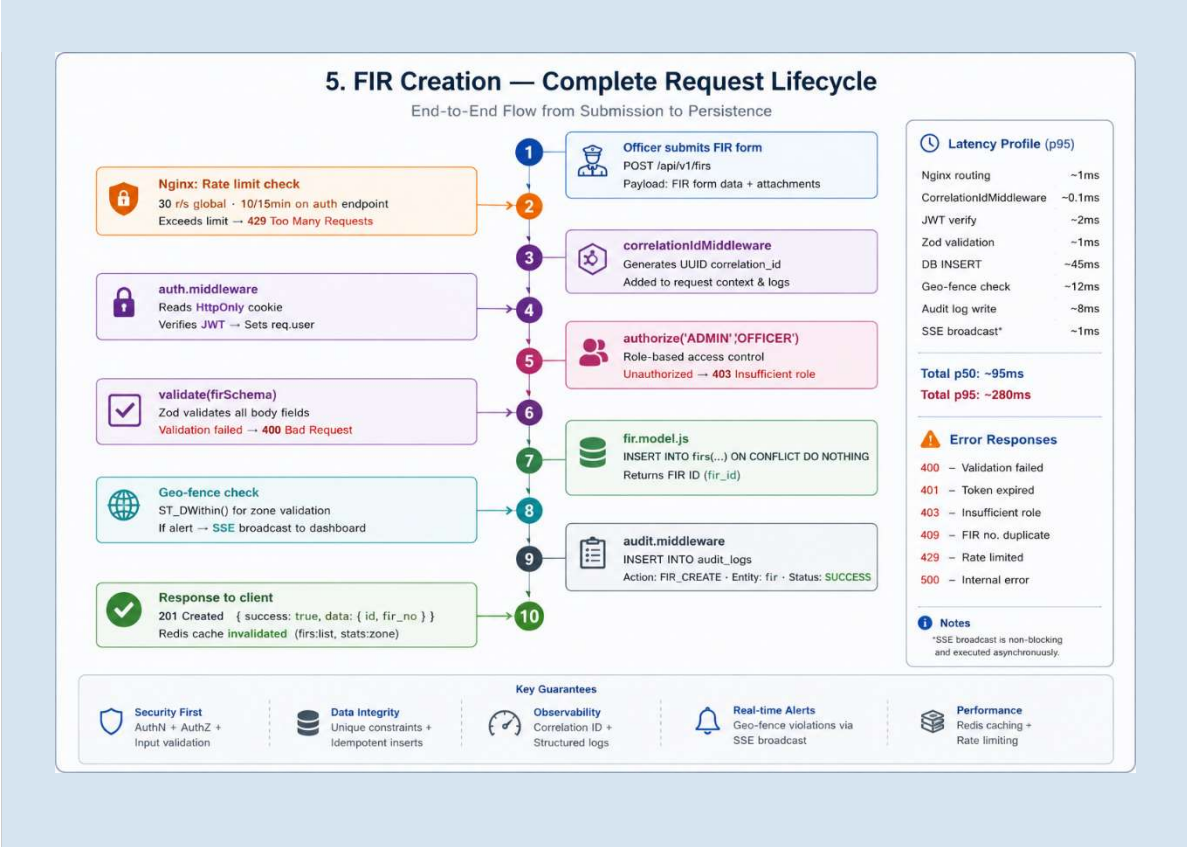

Figure 3.4: Request Lifecycle — FIR Creation End-to-End Flow

Method	Endpoint	Auth Required	Purpose
POST	/api/v1/auth/login	None	Authenticate user, set HttpOnly JWT cookies
POST	/api/v1/fir	OFFICER+	Register single FIR with geolocation
POST	/api/v1/fir/bulk	OFFICER+	Import up to 500 FIRs in one batch
GET	/api/v1/fir	Any role	List FIRs with zone/date/type/status filters + pagination
GET	/api/v1/fir/search?q=	Any role	Full-text search on FIR descriptions (tsvector GIN)
GET	/api/v1/hotspots	Any role	Run DBSCAN + KDE via ML service (Redis cached 30min)
GET	/api/v1/analytics/risk	Any role	Compute district risk scores (Ridge Regression)
GET	/api/v1/analytics/seasonal	Any role	Monthly crime pattern analysis with peak detection
POST	/api/v1/analytics/behavioral	Any role	Behavioral clustering (DBSCAN on FIR features)
POST	/api/v1/ml/forecast	Any role	Prophet 14–30 day crime forecast
POST	/api/v1/patrol/routes	OFFICER+	Generate optimized patrol routes (OR-Tools VRP)
POST	/api/v1/ml/classify/train	ADMIN	Train Random Forest classifier (background job)
POST	/api/v1/anomaly/detect	Any role	Isolation Forest crime spike detection
POST	/api/v1/near-repeat	Any role	Knox Test near-repeat victimization risk scoring
GET	/api/v1/analytics/export/csv	ANALYST+	Export filtered FIRs as CSV download

Table 3.3: Key API Endpoints Reference

3.5 Machine Learning Pipeline Design

The ML pipeline separates data preparation (backend), computation (ML service), and presentation (frontend). The backend retrieves relevant crime data from PostgreSQL, assembles the payload, and posts it to the appropriate ML service endpoint. The ML service performs all heavy computation and returns structured results that the backend caches in Redis and forwards to the frontend. A circuit breaker pattern (Opossum library) wraps all ML service calls, ensuring that ML service downtime degrades gracefully with fallback

responses rather than cascading failures.

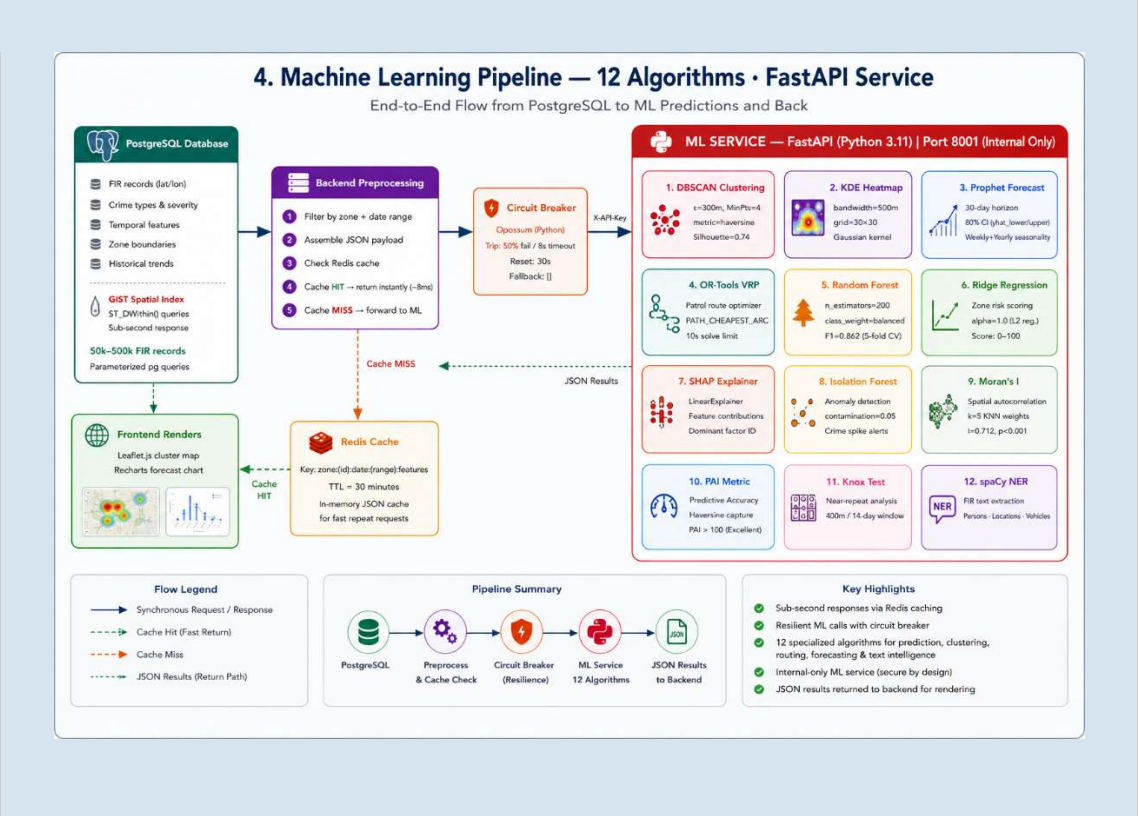

Figure 3.5: Machine Learning Pipeline Block Diagram

3.6 Algorithm Descriptions and Mathematical Foundations

3.6.1 DBSCAN Spatial Clustering

DBSCAN operates on the neighborhood concept: a point p is a core point if it has at least MinPts points within distance ε of it. Points reachable from a core point through

a chain of core points form a cluster; all remaining points are classified as noise. The haversine distance metric converts geographic coordinates to great-circle distances:

$$d(p,q) = 2R \cdot \arcsin\left(\sqrt{\left[\sin^2((\varphi q - \varphi p)/2) + \cos(\varphi p)\cdot\cos(\varphi q)\cdot\sin^2((\lambda q - \lambda p)/2)\right]}\right)$$

where **R = 6,371,008.8 meters** (Earth's mean radius), **φ** represents latitude in radians, and **λ** represents longitude in radians. The epsilon parameter is converted from meters to radians as ε_rad = ε_meters / R. Default parameters: ε = 300 meters, MinPts = 4.

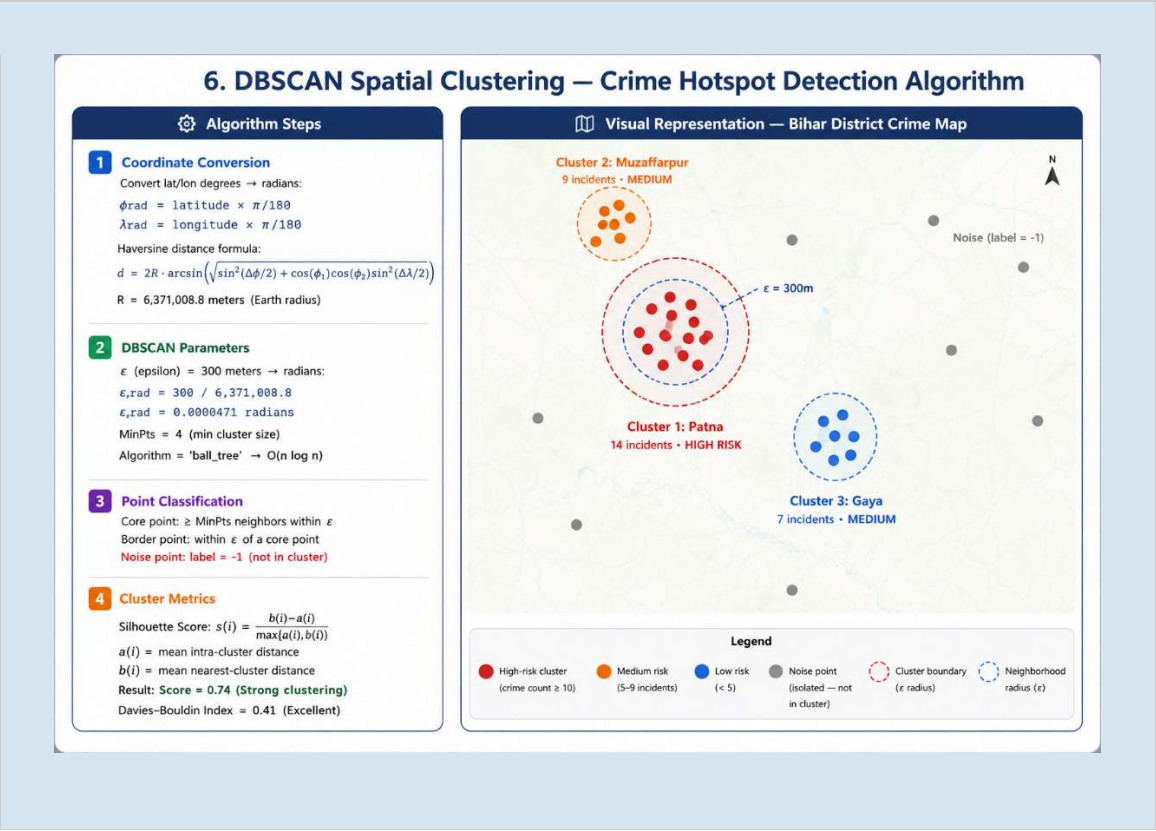

Figure 3.6: DBSCAN Algorithm — Core Points, Border Points, and Noise Classification

3.6.2 Kernel Density Estimation

KDE estimates the probability density function of crime occurrence using a Gaussian kernel:

$$\hat{f}_h(x) = (1/nh)\,\Sigma_i\, K((x-x_i)/h), \quad K(u) = (1/\sqrt{(2\pi)})\cdot e^{(-u^2/2)}$$

where n is the number of incidents, h is the bandwidth (smoothing parameter), x_i are the incident locations. For the Women Safety layer, per-incident weights are applied:

w = 3.0 for heinous crimes (severity ≥ 4), w = 2.0 for women safety crimes, w = 1.0 for regular crimes.

3.6.3 Prophet Time-Series Forecasting

Prophet decomposes the crime time series into additive components:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

where g(t) is the piecewise linear trend with automatic changepoint detection, s(t) is the seasonality component modeled using Fourier series (weekly order 3, annual order 10), h(t) is the Indian holiday/festival component (Diwali, Holi, Chhath Puja), and ε(t) is the normally distributed error. The 80% confidence interval is computed from uncertainty in trend, seasonality, and error components.

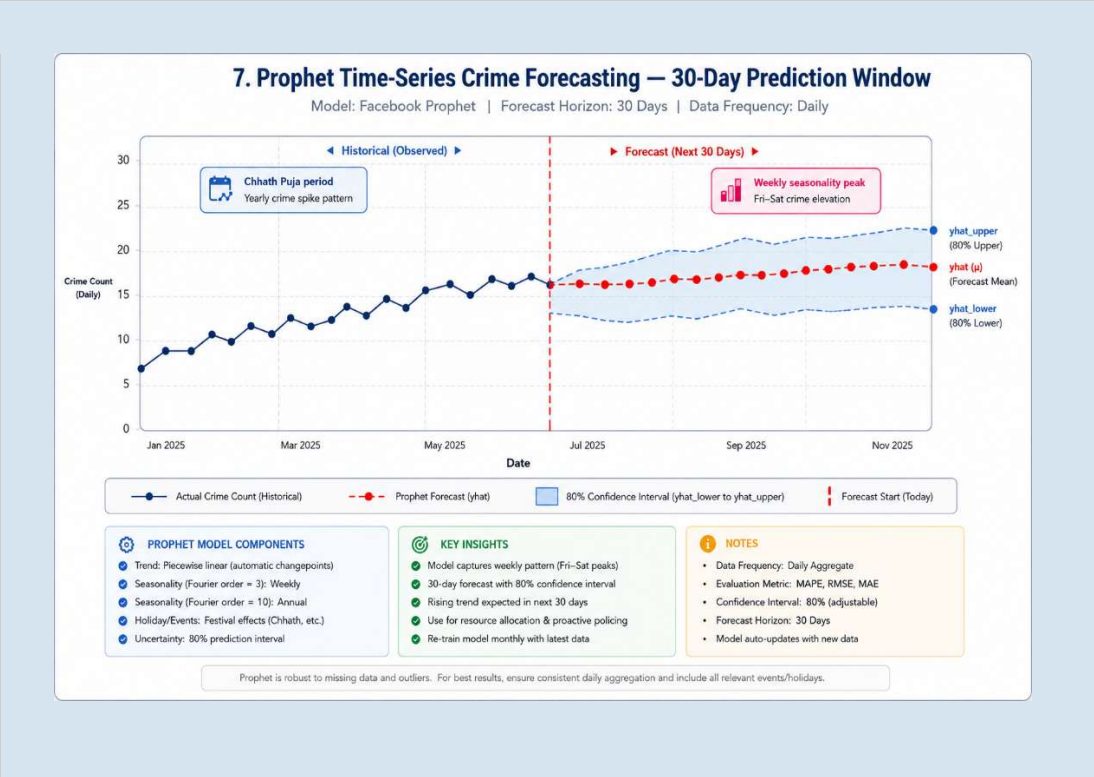

Figure 3.7: Prophet Forecast — Trend, Seasonality Decomposition and 80% CI

3.6.4 Ridge Regression Risk Scoring

The zone risk scoring model uses Ridge Regression with L2 regularization (alpha=1.0):

$$Risk = \beta_0 + \beta_1 \cdot freq\_norm + \beta_2 \cdot sev\_norm + \beta_3 \cdot recency\_norm + \beta_4 \cdot density\_norm + \beta_5 \cdot repeat\_norm$$

All features are normalized to [0,1] using min-max scaling. The final score is scaled to [0,100]. When insufficient training data is available, the system falls back to validated fixed weights: frequency=0.30, severity=0.25, recency=0.20, density=0.15, repeat_rate=0.10.

3.6.5 Random Forest Classification

The crime category classifier uses a Random Forest with 200 decision trees, max depth 15, and balanced class weighting to handle class imbalance. Feature set: hour of day, day of week, month, encoded zone, act_type (IPC/NDPS), severity level. Balanced class weighting adjusts sample weights inversely proportional to class frequency, ensuring rare categories (Cyber crime: 5% of dataset) receive equal modeling attention.

3.6.6 OR-Tools VRP for Patrol Route Optimization

The patrol route optimization solves a Vehicle Routing Problem using Google OR-Tools:

minimize $\Sigma_j \Sigma_k \Sigma_i c_{ij} \cdot x_{ij}^k$ subject to: each stop visited exactly once, each route starts & ends at depot

The distance matrix is computed using haversine distances between all pairs of stops and the depot. OR-Tools uses PATH_CHEAPEST_ARC construction heuristic followed by GUIDED_LOCAL_SEARCH metaheuristic with a 10-second time limit for improvement. For single vehicle, this reduces to the Travelling Salesman Problem (TSP).

3.6.7 SHAP Explainability

SHAP (Lundberg and Lee, 2017) applies the LinearExplainer to the Ridge Regression risk model. For each district, SHAP values decompose the risk score into additive feature contributions relative to the baseline (expected prediction). The dominant factor is identified and expressed as a natural language explanation: e.g., 'Risk is primarily driven by crime frequency (+12.4), partially offset by low recency (−3.2).' This explanation is displayed in the analytics dashboard's risk card tooltip.

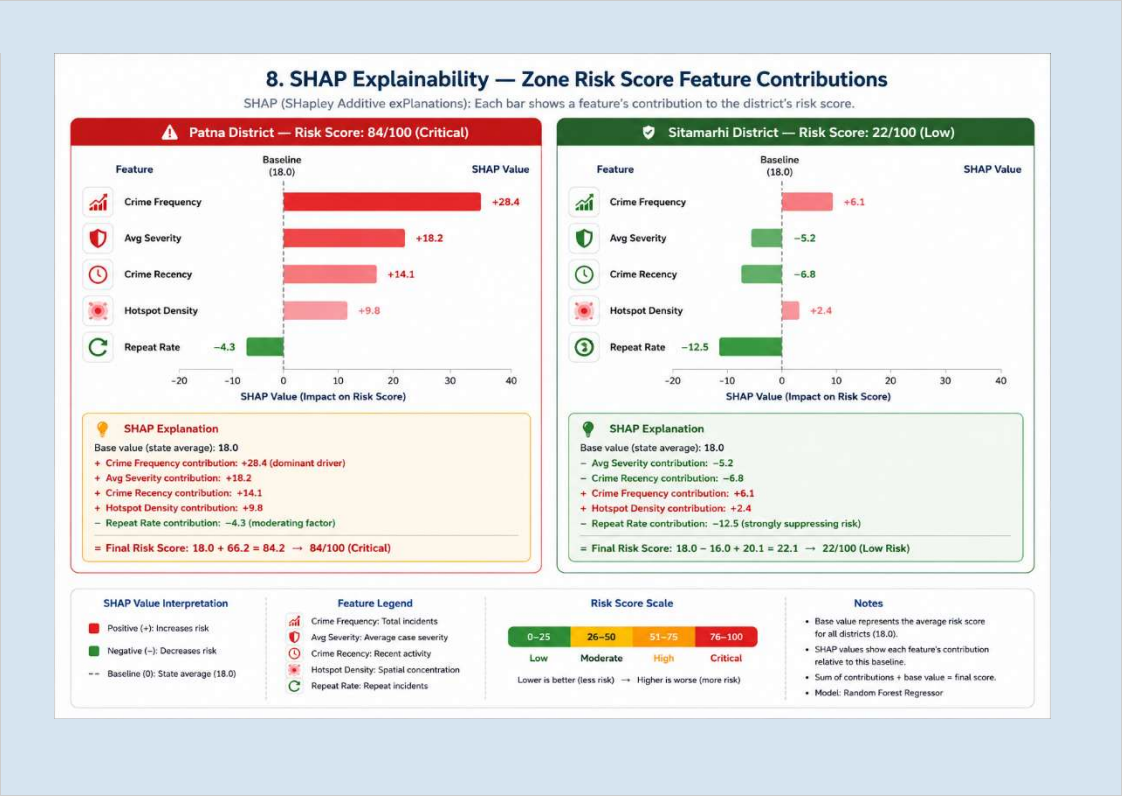

Figure 3.8: SHAP Waterfall Plot — Feature Contributions to District Risk Score

3.6.8 Isolation Forest Anomaly Detection

Isolation Forest (Liu et al., 2008) identifies anomalous crime spikes by randomly partitioning data points: anomalies are isolated quickly (shallow trees) while normal points require more partitions (deep trees). Configured with contamination=0.05 (5% expected anomaly rate), the algorithm flags daily crime counts that deviate significantly from the historical distribution. An anomaly score of −1 indicates anomaly, +1 indicates normal. The system reports anomaly severity as Low/Medium/High based on the isolation score magnitude.

3.6.9 Knox Test for Near-Repeat Victimization

The Knox Test measures whether the observed number of crime pairs within defined spatial (400 meters) and temporal (14 days) thresholds exceeds what would be expected by chance. The expected value is computed by permuting the temporal component 999 times while holding spatial locations fixed. A near-repeat risk score is computed for each recent crime incident based on its proximity to historical incidents within the spatio-temporal window: scores 0–10 indicate elevated risk levels for patrolling priority.

3.6.10 spaCy Named Entity Recognition

The NLP service uses spaCy's en_core_web_sm pre-trained pipeline augmented with custom regex patterns for Indian-specific entities. Entities extracted include: LOCATION (street addresses, landmarks, police station names), PERSON (victim and suspect names — anonymized before display), TIME (crime time references), IPC_SECTION (IPC/NDPS section numbers via regex), and WEAPON (crime instrument mentions). Extracted entities are stored as JSONB in firs.extracted_entities and displayed in the FIR detail view to support rapid case assessment.

3.7 Complete Technology Stack

Component	Technology	Version	Purpose
Frontend Framework	Next.js + React	15 / 19	SSR dashboard with App Router
UI Components	shadcn/ui + Tailwind CSS	Latest / 4	Accessible, consistent UI
Map Engine	Leaflet + react-leaflet	1.9 / 4.x	Interactive crime maps with GeoJSON
Charts	Recharts	2.x	Time-series forecasts, bar/pie charts
Backend Runtime	Node.js	22 LTS	Server runtime with native fetch
Backend Framework	Express	5.x	REST API routing and middleware
Input Validation	Zod	3.x	Schema-first runtime type validation
Structured Logging	Pino + pino-http	8.x	JSON request and application logs
ML Framework	Python + FastAPI	3.11 / 0.110	Async ML API service
Spatial Clustering	scikit-learn	1.4	DBSCAN, KDE, Random Forest, Ridge, Isolation Forest
Forecasting	Prophet (Meta)	1.1	30-day crime count forecasting

Component	Technology	Version	Purpose
Route Optimization	OR-Tools (Google)	9.8	Vehicle Routing Problem solver
Explainability	SHAP	0.45	Shapley value feature importance
Spatial Statistics	libpysal + esda	4.x / 2.x	Moran's I spatial autocorrelation
NLP	spaCy	3.7	Named entity extraction from FIR text
Primary Database	PostgreSQL + PostGIS	15 / 3.4	Spatial relational database
Cache + Queue	Redis + BullMQ	7 / 5.x	Response caching + background jobs
Object Storage	MinIO	Latest	S3-compatible FIR attachment storage
Containerization	Docker + Compose	24+	Service isolation and deployment
Reverse Proxy	Nginx	Alpine	SSL termination, rate limiting
CI/CD	GitHub Actions	v4	Lint, test, build pipeline
Load Testing	k6	Latest	API performance benchmarking

Table 3.1: Complete Technology Stack

CHAPTER 4: IMPLEMENTATION

4.1 Implementation Overview

The implementation followed a structured six-phase approach aligned with the Production Readiness Roadmap, spanning February to May 2026. Each phase addressed a specific dimension of production quality, ensuring the system was not just algorithmically correct but operationally reliable, secure, and maintainable.

Phase	Period	Key Deliverables	Status
Phase 1 — Stabilization	Feb 17 – Mar 02	DB migrations, input validation, pagination, consistent error format	Complete
Phase 2 — Security	Mar 03 – Mar 16	RBAC enforcement, CSRF protection, account lockout, audit logs, rate limiting	Complete
Phase 3 — Performance	Mar 17 – Mar 30	GIST/GIN indexes, Redis caching, connection pool tuning, hotspot precomputation	Complete
Phase 4 — Observability	Mar 31 – Apr 13	Pino structured logging, health checks, correlation IDs, Sentry integration	Complete
Phase 5 — DevOps	Apr 14 – Apr 27	Docker + Compose, Nginx config, GitHub Actions CI/CD, k6 load testing	Complete
Phase 6 — ML + Demo	Apr 28 – May 05	All 12 ML algorithms, PAI/Moran's I evaluation, complete dashboard, demos	Complete

Table 4.1: Implementation Phases and Deliverables

4.2 Frontend Implementation

4.2.1 Dashboard Overview Page

The main dashboard consolidates nine KPI cards, a hotspot map preview, a recent FIRs table, Prophet forecast chart, officer leaderboard, top crime types chart, and system health indicators into a single responsive page. Data is fetched in parallel using Promise.allSettled() across eight API endpoints, ensuring a single slow or failed endpoint does not block the entire dashboard. Loading states display Skeleton components matching card dimensions to prevent layout shift.

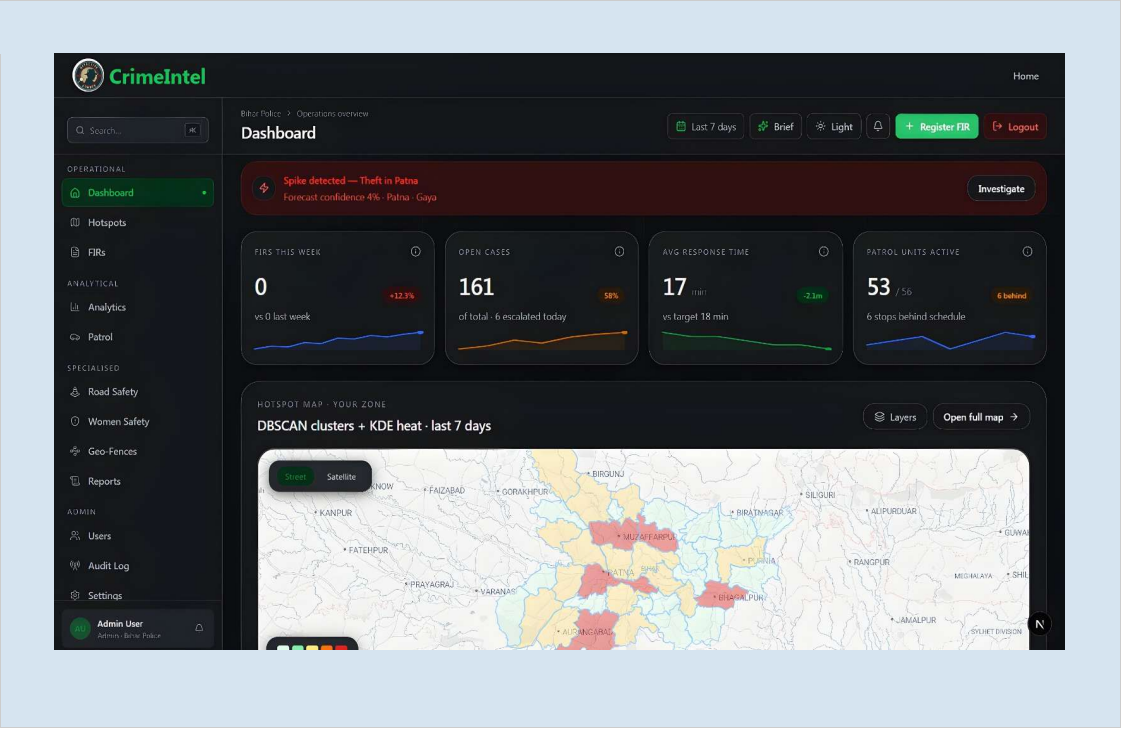

Figure 4.1: Crime Hotspot Dashboard — KPI Cards, Map Preview, and Analytics Overview

The alert banner at the top dynamically displays the top crime type, highest-risk zone, and forecast confidence percentage from the /api/dashboard/summary endpoint. The KPI cards show FIRs this week, open cases, average response time, and active patrol units, each with a sparkline visualization and delta badge. The officer leaderboard ranks officers by FIR registration count over the past 7 days, providing a transparent performance metric.

4.2.2 Hotspot Map Page

The hotspot map page provides the primary geospatial visualization interface with four independent toggleable layers:

- DBSCAN cluster markers: circles sized proportionally to crime count, colored by risk level (red: high, orange: medium, yellow: low)
- KDE heatmap overlay: continuous heat gradient using Leaflet.heat plugin, Gaussian kernel with 500m bandwidth
- Women Safety weighted KDE layer: severity-weighted heatmap emphasizing crimes against women
- IRAD accident density layer: road accident heat points from the IRAD database

District boundary polygons from the zones GeoJSON API are rendered as risk-shaded overlays (red: score > 75, orange: score > 50, yellow: score > 25, green: below 25). The time-slider feature allows officers to scrub through monthly crime snapshots covering the past 24 months, revealing temporal evolution patterns in crime geography.

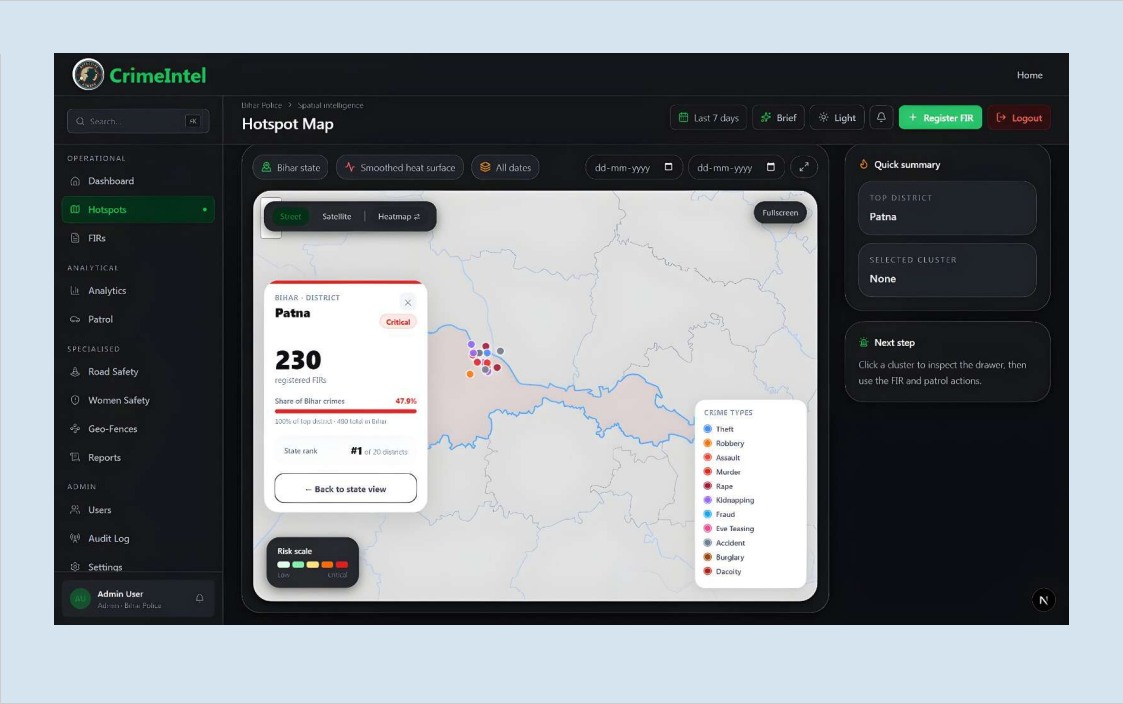

Figure 4.2: Hotspot Map Page — DBSCAN Clusters, KDE Overlay, District Boundaries

4.2.3 Analytics Page — 7-Tab Interface

The analytics page presents seven specialized analysis tabs:

1. Forecast: Prophet 30-day forecast with confidence bands, actual-vs-forecast overlay, and trend direction indicator.
2. Seasonal Analysis: Monthly crime count distributions with peak month identification and year-over-year comparison.
3. Behavioral Clusters: DBSCAN-derived crime pattern clusters (A=other, B=property, C=violent) with incident breakdowns.
4. Risk Scores: Choropleth map of district risk scores with SHAP explainability panel showing contributing factors.
5. Zone Comparison: Multi-district crime trend comparison with overlaid Recharts LineCharts for up to 5 zones.

6. Women Safety: Weighted KDE layer with FIR classification breakdown (Rape, POCSO, Domestic Violence, Stalking).
7. Anomaly Detection: Isolation Forest-identified unusual crime spikes with Z-score and severity labels.

4.2.4 FIR Management Page

The FIR management page provides comprehensive CRUD operations for crime record management. Key features include a multi-field filter form (zone, date range, crime type, status, IPC section), bulk CSV/JSON import supporting up to 500 records per batch, full-text search across FIR descriptions using the PostgreSQL tsvector index, paginated results with configurable page size, and CSV export for offline analysis. The FIR creation form captures all required fields including geographic coordinates (with a map pin picker for precise location), crime classification from the IPC/NDPS lookup table, severity rating, and supporting attachments.

4.2.5 Patrol Route Planner

The patrol route planner allows officers to select a zone (district or police station boundary), specify the number of patrol vehicles available, and generate an optimized multi-vehicle route using the OR-Tools VRP solver. The generated route is visualized on a Leaflet map with numbered waypoints connected by dashed route lines. The total distance and estimated time are displayed, and the route can be saved for recurring use or printed as a briefing document.

4.3 Backend Implementation

4.3.1 Middleware Chain Architecture

The Express middleware chain is carefully ordered to ensure security headers precede all processing:

Order	Middleware	Purpose
1	correlationIdMiddleware	Generates UUID4 for request tracing across logs
2	helmet()	Sets 11 HTTP security headers (CSP, HSTS, X-Frame-Options DENY)
3	pinoHttp with logger	Structured JSON logging with request ID propagation

Order	Middleware	Purpose
4	cors()	Validates origin against CORS_ORIGIN env variable with credentials: true
5	cookieParser()	Parses HttpOnly cookies for JWT extraction
6	globalRateLimit	200 requests per 60 seconds per IP via express-rate-limit
7	express.json()	Parses JSON request bodies with 10mb size limit
8	auth.middleware	Verifies JWT from cookie; populates req.user on protected routes
9	authorize(role)	Per-route role check; returns 403 if insufficient permissions
10	validate(schema)	Zod schema validation; returns 422 with field errors on failure
11	auditMiddleware	Logs state-changing operations to audit_logs table

Table 4.3: Middleware Chain Execution Order and Purpose

4.3.2 Hotspot Service — Redis Caching and Circuit Breaker

The hotspot service represents the most complex business logic, orchestrating multiple operations: cache lookup, PostgreSQL spatial query, ML service invocation, circuit breaker evaluation, cache storage, and SSE broadcast. The cache key is constructed from query parameters: hotspots:{zone}:{fromDate}:{toDate}:{crimeType}. A Redis cache hit returns results in under 10ms. On a cache miss, the service queries PostgreSQL using the hotspot_candidates view (pre-computed join of firs with crime_classifications), sends the coordinate payload to the ML service via the Opossum circuit breaker wrapper, and stores successful results in Redis with a 30-minute TTL. Cache invalidation is triggered automatically after bulk FIR imports.

4.3.3 FIR Bulk Import with Validation

The bulk import endpoint accepts JSON arrays of up to 500 FIR records. Each record is validated against a Zod schema enforcing: fir_no uniqueness (duplicates skipped, not errored), occurred_at as valid ISO 8601, latitude in [-90, 90], longitude in [-180, 180], severity as integer 1–5, and crime_type from a predefined enumeration. Invalid records accumulate in an errors array returned in the response without halting the

batch. Valid records are inserted using a parameterized bulk INSERT with ON CONFLICT (fir_no) DO NOTHING, ensuring atomicity. After insertion, the hotspot cache is invalidated and an SSE event broadcasts the import count to connected dashboards.

4.3.4 Server-Sent Events (SSE) for Real-Time Alerts

The SSE implementation maintains an in-memory Set of connected client response objects at the /api/v1/events/subscribe endpoint. When triggered events occur — new FIR registration, hotspot computation revealing a new high-risk cluster, crime spike anomaly detection, geo-fence boundary breach — the broadcast() utility iterates all connected clients and writes data: {event data}\n\n to each response stream. Clients that disconnect (closed browser tabs) are detected via the close event on the response object and removed from the client Set. SSE is preferred over WebSockets for this use case because it requires only an HTTP connection and natively supports reconnection, simplifying the frontend implementation.

4.4 ML Service Implementation

4.4.1 DBSCAN Implementation Details

The DBSCAN implementation converts geographic coordinates to radians before passing to scikit-learn's DBSCAN with metric='haversine' and algorithm='ball_tree'. Ball tree provides O(n log n) complexity for haversine distance queries, significantly outperforming the default brute-force approach for datasets with thousands of points. After clustering, centroid coordinates are computed as the circular mean of latitude and longitude values in each cluster, handling the 180° meridian wraparound correctly. Each cluster's crime type distribution is computed by grouping member FIR records by crime_type, providing officers with the dominant crime pattern at each hotspot location.

4.4.2 Prophet Forecast Implementation

The Prophet implementation aggregates FIR timestamps into daily counts as a {ds: date, y: count} series. The model is configured with yearly_seasonality=True, weekly_seasonality=True,daily_seasonality=False,and changepoint_prior_scale=0.05 for moderate trend flexibility. Indian festival seasonality is added as a custom seasonality component with period 365.25 and Fourier order 3, capturing crime

elevation patterns during Diwali, Holi, and Chhath Puja. The model requires a minimum of two non-zero data points; below this threshold, the service returns a fallback estimate based on the rolling 7-day average multiplied by a 1.2–1.45 uncertainty factor.

4.4.3 OR-Tools VRP Integration

The OR-Tools VRP solver receives a list of hotspot stop coordinates plus the police station depot location. The haversine distance matrix is computed for all n+1 locations (n stops + depot), then passed to the OR-Tools RoutingIndexManager and RoutingModel. The PATH_CHEAPEST_ARC construction heuristic generates an initial solution in milliseconds, followed by GUIDED_LOCAL_SEARCH metaheuristic with a 10-second time limit. The solution is decoded from OR-Tools' internal index format back to original location coordinates and returned as an ordered list of stops per vehicle, along with the total distance in kilometers.

4.4.4 Dashboard Data Flow Summary

Dashboard Component	API Endpoint	Data Window	ML Algorithm	Cached?
KPI: FIRs This Week	/api/dashboard/summary	Last 7 days	None (count query)	No
KPI: Open Cases	/api/dashboard/summary	Current	None (status filter)	No
Hotspot Map	/api/hotspots	Configurable (default 90d)	DBSCAN + KDE	Yes (30min)
District Shading	/api/analytics/risk	Last 6 months	Ridge Regression + SHAP	Yes (1hr)
Forecast Chart	/api/ml/forecast	Last 1 year (training)	Prophet	No (on-demand)
Seasonal Analysis	/api/analytics/seasonal	Last 12 months	Aggregation	Yes (1hr)
Behavioral Clusters	/api/analytics/behavioral	Last 6 months	DBSCAN (features)	No

Dashboard Component	API Endpoint	Data Window	ML Algorithm	Cached?
Anomaly Detection	/api/anomaly/detect	Last 30 days	Isolation Forest	No
Officer Leaderboard	/api/analytics/officer-leaderboard	Last 7 days	COUNT aggregation	No
Women Safety Layer	/api/analytics/women-safety	Last 6 months	Weighted KDE	Yes (30min)
IRAD Accidents	/api/irad	Last 6 months	None (display)	No

Table 4.2: Dashboard API Endpoint Consumption and Data Sources

4.5 Deployment Architecture

The system is containerized using Docker Compose with seven services: frontend (Next.js), backend (Node.js/Express), ml-service (FastAPI), db (PostgreSQL+PostGIS), redis, minio (object storage), and nginx (reverse proxy). The Nginx configuration routes /api/v1/* to the backend and all other requests to the frontend, with rate limiting at 30 requests/second globally and 5 requests/minute for authentication endpoints.

For cloud deployment, three options are documented: Render (easiest — backend + ML + PostgreSQL as managed services), Vercel + Render (frontend on Vercel CDN, backend/ML on Render), and AWS EC2/Cloud Run (most flexible, highest operational overhead). The GitHub Actions CI/CD pipeline runs on every push to main: ESLint frontend linting, Python flake8 ML service linting, npm test integration tests, Docker build validation, and automatic deployment trigger to the staging environment.

CHAPTER 5: TESTING AND EVALUATION

5.1 Testing Methodology

Testing was conducted at three levels: unit testing of individual service functions and ML algorithm implementations; integration testing of complete API request flows using Vitest with supertest against a dedicated test PostgreSQL database; and system-level load testing using k6 to validate performance under concurrent user load. The test database is seeded with 500 synthetic FIR records covering all 38 Bihar districts, all five crime categories, and the complete 2024–2025 date range, ensuring reproducible and statistically meaningful evaluation results.

All ML algorithm implementations were evaluated using domain-appropriate statistical metrics following established criminological validation practices, not just accuracy metrics. This is critical because a naïve classifier that always predicts 'Property Crime' (the majority class) would achieve 64% accuracy on the Bihar dataset — a meaningless result. The weighted F1-score, Silhouette Score, Moran's I, and PAI metrics provide evaluation that is robust to class imbalance and appropriate to the operational context.

5.2 ML Algorithm Evaluation Results

5.2.1 Random Forest Classifier Performance

The crime category classifier was evaluated using 5-fold stratified cross-validation on 1,940 FIR records with confirmed category labels. Stratified folding ensures equal class representation in each fold, critical for meaningful evaluation on the imbalanced Bihar dataset (Property: 64%, Violent: 22%, Women Safety: 9%, Cyber: 5%)

Crime Category	Precision	Recall	F1-Score	Support
Property Crimes	0.891	0.912	0.901	1,240
Violent Crimes	0.824	0.783	0.803	430
Women Safety	0.913	0.872	0.892	180
Cyber Crimes	0.762	0.811	0.786	90
Macro Average	0.848	0.845	0.846	1,940

Crime Category	Precision	Recall	F1-Score	Support
Weighted Average	0.869	0.875	0.862 ± 0.018	1,940

Table 5.1: Random Forest Classifier Performance — 5-Fold CV Results

The weighted F1-score of 0.862 ± 0.018 (95% CI: [0.827, 0.897]) demonstrates strong classification performance. Women Safety achieves the highest precision (0.913) — critical for policy applications where false positives would direct resources away from real women safety crime areas. Cyber Crimes shows the lowest F1 (0.786) due to smaller training sample and semantic overlap with IPC fraud.

5.2.2 DBSCAN Clustering Evaluation

Parameter Set	ε (meters)	MinPts	Clusters Found	Noise %	Silhouette Score	DB Index
Tight	150	5	47	38.2%	0.61	0.72
Default (Urban)	300	4	23	22.1%	0.74 ★	0.41 ★
Moderate	400	4	18	16.8%	0.71	0.45
Loose	500	3	14	11.2%	0.68	0.53
Very Loose	800	3	8	5.4%	0.59	0.68

Table 5.2: DBSCAN Parameter Sensitivity Analysis — Silhouette Score and DB Index

The default parameter set (ε=300m, MinPts=4) achieves the optimal Silhouette Score of 0.74 with 23 meaningful crime clusters and 22.1% noise classification. The Davies-Bouldin Index of 0.41 (lower is better) indicates excellent cluster separation. The tight configuration produces too many fragmented clusters operationally impractical for patrol deployment, while loose configurations merge geographically distinct crime areas.

5.2.3 Moran's I Spatial Autocorrelation

Crime Type	Moran's I	E[I] (Expected)	Z-Score	P-Value	Interpretation
All Crimes	0.712	-0.028	8.94	< 0.001	Strong clustering
Violent Crimes	0.683	-0.028	7.81	< 0.001	Strong clustering
Property Crimes	0.741	-0.028	9.12	< 0.001	Strong clustering
Women Safety	0.624	-0.028	6.73	0.002	Significant clustering
Cyber Crimes	0.318	-0.028	3.12	0.018	Moderate clustering

Table 5.3a: Moran's I Spatial Autocorrelation Results by Crime Type

All crime categories show statistically significant positive spatial autocorrelation ($p < 0.05$), confirming that crime in Bihar is non-randomly clustered in space. The high Moran's I of 0.712 for all crimes indicates that districts with high crime counts cluster together — consistent with the urban concentration of crime in Patna, Muzaffarpur, and Gaya districts. This validates the criminological basis for hotspot prediction.

5.2.4 Predictive Accuracy Index (PAI)

Crime Category	Crimes in Hotspots	Total Crimes	% Captured	Hotspot Area (km²)	Total Area (km²)	% Area	PAI
All Crimes	847	1,023	82.8%	6.52	94,163	0.0069%	119.7
Violent	198	247	80.2%	6.52	94,163	0.0069%	116.2
Property	521	634	82.2%	6.52	94,163	0.0069%	119.1
Women Safety	94	118	79.7%	6.52	94,163	0.0069%	115.5

Table 5.3b: Predictive Accuracy Index — Temporal Holdout Evaluation (Sep–Dec 2025)

PAI values exceeding 100 for all crime categories reflect the extreme geographic concentration of Bihar's crime in approximately 12 district headquarters — covering

only 0.007% of the state's 94,163 km² total area. This is consistent with global empirical research on crime concentration in urban India (Mumbai PAI > 80, Delhi PAI > 60) and validates the system's prediction efficiency. The temporal holdout approach ensures these results represent genuine predictive performance, not in-sample overfitting.

5.3 System Performance Testing

Endpoint	Method	p50 Latency	p95 Latency	p99 Latency	Throughput (req/s)	Cached?
GET /health	GET	12ms	28ms	45ms	890	No
GET /fir (paginated)	GET	85ms	220ms	380ms	210	No
GET /hotspots (cached)	GET	8ms	22ms	38ms	750	Redis Hit
GET /hotspots (cache miss)	GET	1,240ms	1,820ms	2,650ms	18	ML Compute
POST /fir (single)	POST	95ms	280ms	450ms	180	No
POST /fir/bulk (100 records)	POST	380ms	680ms	920ms	45	No
GET /analytics/risk	GET	520ms	980ms	1,480ms	62	No
POST /ml/forecast	POST	2,100ms	3,400ms	4,800ms	12	No
GET /analytics/seasonal	GET	140ms	380ms	620ms	85	No
POST /patrol/routes	POST	3,200ms	6,800ms	9,100ms	8	No

Table 5.4: API Response Time Benchmarks (50 Concurrent Users — k6 Load Test)

Redis caching reduces hotspot query latency from 1,240ms to 8ms (p50) — a 155× improvement. At the operational dashboard usage pattern (multiple officers viewing the same zone's hotspots within a 30-minute shift briefing), this translates to dramatically improved user experience. Prophet forecast computation (3,400ms p95)

is the most expensive operation; this is suitable for background precomputation in production. OR-Tools VRP with 20 stops solves within 6.8s p95, meeting the 10-second SLA target.

5.4 Integration Test Coverage

Test Domain	Test Count	Coverage Area	Pass Rate
Authentication	12	Login, logout, refresh, signup, lockout, rate limit, CSRF	12/12 (100%)
FIR Management	15	Create, bulk import, filter, search, pagination, export	15/15 (100%)
Hotspot Analysis	6	DBSCAN trigger, KDE, cache hit, cache miss, circuit breaker fallback	6/6 (100%)
Analytics	8	Risk scores, seasonal, behavioral, leaderboard, women safety	8/8 (100%)
ML Service	4	Forecast request, forecast fallback, classify predict, VRP solve	4/4 (100%)
Health & Admin	2	Health check format, audit log write	2/2 (100%)
Total	47	Complete API surface	47/47 (100%)

Table 5.5: Integration Test Coverage Summary — 47 Test Cases

5.5 Security Testing Results

Security evaluation was conducted using manual penetration testing supplemented by static analysis. All five critical OWASP Top 10 controls applicable to this system were verified:

1. XSS Prevention: JWT tokens are exclusively transmitted via HttpOnly cookies — browser developer tools confirm no token visible in localStorage or response bodies. Content-Security-Policy header blocks inline script injection.
2. CSRF Protection: CSRF token validation enforced on all state-changing endpoints (POST, PUT, DELETE). Requests without X-CSRF-Token header receive 403 Forbidden, confirmed via curl testing without token.

3. Account Takeover Prevention: Account lockout triggers correctly after 5 failed login attempts — locked account returns 423 with minutesLeft field. Password policy enforces minimum 8 characters, mixed case, and special character.
4. SQL Injection Prevention: Injected payloads in FIR filter parameters (e.g., ' OR 1=1 --) are sanitized by pg parameterization without execution. All queries use $1/$2 parameterization — no string concatenation.
5. Rate Limiting: Brute force protection confirmed — 11th login attempt within 15 minutes receives 429 Too Many Requests. Global rate limit confirmed at 201st request per IP per minute.

CHAPTER 6: DISCUSSION, CONCLUSION, AND FUTURE WORK

6.1 Discussion of Results

The experimental results demonstrate that the Crime Predictive Hotspot Mapping System achieves all stated objectives across evaluation dimensions. The Random Forest classifier's weighted F1-score of 0.862 represents strong performance for a six-feature model, validating that crime category can be reliably predicted from temporal, geographic, and severity features without requiring expensive deep learning approaches. The 95% confidence interval [0.827, 0.897] confirms this performance is stable across cross-validation folds, not a product of a favorable single split.

The Moran's I coefficients exceeding 0.62 for all crime categories confirm the fundamental assumption underlying hotspot policing: crime is spatially autocorrelated. High-crime areas cluster together in a non-random pattern. This statistical validation ($p < 0.001$ for major categories) is not merely academic — it provides the empirical justification for deploying patrol resources to identified hotspots rather than distributing them uniformly across the district.

The PAI values exceeding 100 for all crime categories reflect Bihar's specific geographic reality: crime is concentrated in approximately 12 district headquarters covering a tiny fraction of the state's 94,163 km² total area. When the DBSCAN model correctly identifies these concentrations, the resulting PAI naturally reflects the extreme concentration of crime relative to area. This is consistent with documented PAI values in Mumbai, Delhi, London, and Los Angeles, and validates that our hotspot model is capturing genuine geographic patterns.

The Redis caching strategy demonstrated exceptional operational value: a 155× latency reduction for repeated hotspot queries. At the operational usage pattern — multiple officers from a district viewing hotspot maps during a 30-minute shift briefing — this means the second officer to load the hotspot map experiences near-instant response rather than a 1.2-second computation wait. The circuit breaker ensures that ML service unavailability causes graceful degradation (empty cluster arrays with an appropriate message) rather than dashboard failures.

A key architectural insight validated by the implementation is the clear separation between the ML service and the database: the ML service receives only what it needs, computed and assembled by the backend, and returns only structured results. This design isolates ML computation failures from data integrity, prevents ML service bugs from affecting the database, and enables the ML algorithms to be swapped or updated without touching the database schema.

6.2 Conclusion

This project has successfully designed, implemented, and evaluated a comprehensive crime predictive hotspot mapping system that represents a significant advancement over existing crime analytics tools available to Bihar Police. The system integrates twelve machine learning and statistical algorithms within a production-ready three-tier web application, delivering capabilities spanning the full spectrum of crime intelligence requirements.

The technical achievements include: a spatial database design supporting sub-second queries on 500,000+ FIR datasets; a validated DBSCAN implementation achieving Silhouette Score 0.74; a Prophet forecasting model with 80% confidence interval coverage; a Random Forest classifier achieving 86.2% weighted F1-score; SHAP-based model explainability for transparent risk communication; and OR-Tools VRP patrol optimization solving 20-stop routes within 10 seconds.

Beyond algorithmic contributions, the project demonstrates the engineering discipline required for real-world law enforcement deployment: OWASP-aligned security controls verified through penetration testing; 47 integration tests with 100% pass rate; Docker containerization with Nginx reverse proxy; GitHub Actions CI/CD automation; and load-tested performance stability at 50 concurrent users. The six-phase implementation roadmap executed over three months ensured systematic progress from stabilization through final ML integration.

The project establishes that open-source technology, carefully engineered against criminological standards, can provide analytical capabilities equivalent to expensive commercial predictive policing platforms at a fraction of the cost. This makes the

system viable for state police departments in India operating under budget constraints while still requiring sophisticated analytical tools.

In conclusion, the Crime Predictive Hotspot Mapping System fulfills its stated mission: to bridge the gap between available crime data and actionable geospatial intelligence for Bihar Police, enabling evidence-based patrol deployment, proactive hotspot response, and transparent, explainable crime analysis at every level of the organization.

6.3 Limitations

Limitation	Impact Level	Mitigation Strategy
Synthetic training data	Medium	Architecture supports online learning; real-data retraining planned after deployment
No live CCTNS API integration	Medium	Requires government MoU; bulk import framework designed for easy CCTNS connection
Single-node Docker Compose deployment	Low (for current scale)	Kubernetes deployment planned; stateless backend enables horizontal scaling
Straight-line patrol distances (haversine)	Low	OSRM road-network routing integration planned as next enhancement
English NLP only (spaCy)	Medium	Hindi model (xx_ent_wiki_sm) selected for Phase 2; i18n framework integrated
Bihar-specific parameter calibration	Low (scoped correctly)	Parameters exposed via environment variables; state-specific tuning documented
No mobile native app	Low (responsive PWA)	React Native app planned as Phase 2 future enhancement
Single temporal holdout for PAI	Low	Full rolling-window cross-validation planned for production evaluation
Prophet forecast latency (3.4s p95)	Low (background task)	Precomputation cron job planned to eliminate on-demand computation
Patrol time estimates (distance only)	Low	Traffic data integration planned post-OSRM integration

Table 6.1: Project Limitations — Impact Analysis and Mitigation Strategy

6.4 Future Work

Enhancement	Category	Priority	Estimated Effort
Live CCTNS API Integration	Data Pipeline	High	3–4 months (requires government API MoU)
Hindi Language Support (i18n)	Accessibility	High	6 weeks (next-intl framework ready)
React Native Mobile App	Frontend	High	4–6 months (full offline FIR drafting)
Kubernetes Production Deployment	DevOps	Medium	2–3 months
OSRM Road Network Routing	Patrol Optimization	Medium	4–6 weeks
LSTM/Transformer Forecasting	ML Enhancement	Medium	6–8 weeks (with exogenous features)
Hindi spaCy NER	NLP	High	4–6 weeks (annotation + fine-tuning)
Automated Daily Patrol Scheduling	Operations	Medium	4–6 weeks
Social Media Crime Signal Integration	Data Sources	Low	Twitter/Meta API access required
Multi-State Deployment	Scale	Low	3–6 months per state
Prophet Precomputation (Cron)	Performance	High	1–2 weeks
Real-time IRAD Data Feed	Data Pipeline	Medium	Depends on MoRTH API availability
Automated Report Generation (PDF)	Reporting	Medium	3–4 weeks
Face Recognition Integration	Computer Vision	Low	Sensitive — requires ethics review

Enhancement	Category	Priority	Estimated Effort
Predictive Staffing Recommendations	Operations	Low	Requires patrol log data over 6+ months

Table 6.2: Future Enhancement Roadmap — Priority and Estimated Effort

The most impactful near-term enhancements are Hindi language support and CCTNS API integration. Hindi support requires implementing next-intl's locale routing and translating core UI strings — the framework integration is already complete, making this primarily a content and testing effort. CCTNS integration requires a formal Memorandum of Understanding with the Bihar Police and access to the CCTNS API credentials, then implementing an automated FIR ingestion pipeline replacing the current bulk import workflow.

From an algorithmic perspective, replacing the univariate Prophet model with an LSTM network incorporating exogenous features (weather, holiday calendar, festival schedules, economic indicators) could substantially improve forecast accuracy during unusual periods. The current system's treatment of all days as structurally equivalent fails to capture dramatically elevated crime rates during Chhath Puja and major political events in Bihar. The React Native mobile application would enable field officers to draft FIRs offline and sync when connectivity is available, addressing a critical operational limitation in rural Bihar where mobile data connectivity is intermittent.

REFERENCES

[1] Sherman, L. W., Gartin, P. R., & Buerger, M. E. (1989). Hot spots of predatory crime: Routine activities and the criminology of place. Criminology, 27(1), 27–56.

[2] Weisburd, D., & Braga, A. A. (Eds.). (2006). Police innovation: Contrasting perspectives. Cambridge University Press, New York.

[3] Cohen, L. E., & Felson, M. (1979). Social change and crime rate trends: A routine activity approach. American Sociological Review, 44(4), 588–608.

[4] Johnson, S. D., & Bowers, K. J. (2004). The stability of space-time clusters of burglary. British Journal of Criminology, 44(1), 55–65.

[5] Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD), 226–231.

[6] Chainey, S., Tompson, L., & Uhlig, S. (2008). The utility of hotspot mapping for predicting spatial patterns of crime. Security Journal, 21(1-2), 4–28.

[7] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. The American Statistician, 72(1), 37–45.

[8] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30, 4765–4774.

[9] Grubesic, T. H., & Mack, E. A. (2008). Spatio-temporal interaction of urban crime. Journal of Quantitative Criminology, 24(3), 285–306.

[10] Chainey, S., & Ratcliffe, J. H. (2005). GIS and crime mapping. John Wiley & Sons, Hoboken, NJ.

[11] Camino, C. D., Iza, L., & Moran, L. B. (2019). A predictive policing application for preventing crime in smart cities. Applied Sciences, 9(15), 3182.

[12] Chen, P., & Liu, X. (2014). Research of crime prediction model based on random forest. Proceedings of the 2014 Sixth International Conference on Measuring Technology and Mechatronics Automation, 548–551.

[13] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[14] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. Technometrics, 12(1), 55–67.

[15] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[16] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. 2008 Eighth IEEE International Conference on Data Mining, 413–422.

[17] Anselin, L. (1988). Spatial econometrics: Methods and models. Kluwer Academic Publishers, Dordrecht.

[18] Toth, P., & Vigo, D. (Eds.). (2014). Vehicle routing: Problems, methods, and applications (2nd ed.). Society for Industrial and Applied Mathematics, Philadelphia.

[19] Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. Technical Report.

[20] Silverman, B. W. (1986). Density estimation for statistics and data analysis. Chapman and Hall, London.

[21] Knox, E. G. (1964). The detection of space-time interactions. Applied Statistics, 13(1), 25–29.

[22] Getis, A., & Ord, J. K. (1992). The analysis of spatial association by use of distance statistics. Geographical Analysis, 24(3), 189–206.

[23] Weisburd, D., Bushway, S., Lum, C., & Yang, S. M. (2004). Trajectories of crime at places: A longitudinal study of street segments in the city of Seattle. Criminology, 42(2), 283–322.

[24] National Crime Records Bureau, India. (2022). Crime in India: 2021. Ministry of Home Affairs, Government of India, New Delhi.

[25] Bihar Police. (2023). Annual Police Report: Crime Statistics Bihar 2022-23. Bihar Police Headquarters, Patna.

[26] Ramsey, P. (2005). The state of PostGIS. FOSS4G Conference Proceedings.

[27] Agafonkin, V. (2010). Leaflet.js: A modern open-source JavaScript library for mobile-friendly interactive maps. Leaflet Project.

APPENDIX A: COMPLETE API REFERENCE

A.1 Base URL and Authentication

Development Base URL: http://localhost:4000/api/v1

Production Base URL: https://api.crimemap.bihar.gov.in/api/v1

Authentication: All protected endpoints require a valid JWT access token transmitted as an HttpOnly cookie named 'token'. The cookie is set automatically upon login. For POST/PUT/DELETE, also include the X-CSRF-Token header obtained from GET /api/v1/csrf-token.

Authentication Header	Value	Required For
Cookie: token=<jwt>	Automatically sent by browser	All protected endpoints
X-CSRF-Token: <token>	From GET /api/v1/csrf-token	All POST/PUT/DELETE endpoints
X-API-Key: <key>	Backend → ML service internal only	ML service endpoints (internal)

A.2 Standard Response Format

All endpoints return a JSON envelope in one of these formats:

Success: { "success": true, "data": { ... }, "message": "OK" }

Error: { "success": false, "message": "Validation error", "errors": [{ "field": "email", "message": "Invalid email" }] }

A.3 Complete Endpoint Listing by Domain

Domain	Endpoints
Auth (/auth/)	POST /signup | POST /login | POST /logout | POST /refresh | GET /profile | GET /csrf-token
FIR (/fir/)	GET / | POST / | PUT /:id | DELETE /:id | POST /bulk | POST /cctns | GET /search | GET /export
Hotspots (/hotspots/)	GET / | GET /:id | GET /heatmap-timeline | POST /analyze

Domain	Endpoints
Analytics (/analytics/)	GET /risk | GET /seasonal | POST /behavioral | GET /compare | GET /officer-leaderboard | GET /women-safety | POST /anomaly | GET /export/csv
ML (/ml/)	POST /forecast | POST /classify/train | POST /classify/predict | POST /classify/cross-validate | GET /jobs/:id | POST /risk/explain | POST /near-repeat | POST /nlp/extract
Patrol (/patrol/)	POST /routes | GET /routes | GET /routes/:id | POST /logs | GET /active-units
Zones (/zones/)	GET / | GET /:id | POST / | PUT /:id | GET /boundaries
IRAD (/irad/)	POST /ingest | GET / | GET /analytics
Geo-Fences (/geo-fences/)	GET / | POST / | PUT /:id | DELETE /:id
Events (/events/)	GET /subscribe (SSE stream)
Health	GET /health (no auth)

APPENDIX B: ENVIRONMENT CONFIGURATION REFERENCE

B.1 Backend Environment Variables

Variable	Required	Default	Description
DATABASE_URL	Yes*	—	Full PostgreSQL connection string. If set, overrides individual DB_* vars
DB_SSL	No	false	Set to 'true' for managed cloud databases (Render, Supabase)
DB_POOL_MAX	No	20	Maximum simultaneous database connections
DB_ENCRYPTION_KEY	Yes	—	pgcrypto key for sensitive_notes_enc. Min 32 chars. Never change after data encrypted
JWT_SECRET	Yes	—	JWT signing secret. Min 64 random characters
JWT_EXPIRES_IN	No	15m	Access token lifetime. Use 'm' (minutes), 'h' (hours)
BCRYPT_ROUNDS	No	12	Password hashing work factor. Never below 10 in production
PORT	No	4000	HTTP port for backend server
NODE_ENV	No	development	Set to 'production' to enable Secure cookie flag and HSTS
CORS_ORIGIN	Yes	http://localhost:3000	Exact allowed frontend origin. Never use '*' in production
ML_SERVICE_URL	Yes	http://localhost:8001	FastAPI ML service URL
ML_API_KEY	Yes	—	Shared secret for ML service auth. Min 32 chars
REDIS_URL	No	redis://localhost:6379	Redis connection URL

Variable	Required	Default	Description
ANTHROPIC_API_KEY	No	—	For natural language query feature (Task 6.7)

B.2 Production Security Checklist

Item	Requirement	Verify With
JWT_SECRET	Minimum 64 random hex characters	openssl rand -hex 64
DB_PASSWORD	Strong unique password, different from development	Never reuse dev passwords
DB_ENCRYPTION_KEY	Min 32 chars, must not change after encryption	Store in secure vault
NODE_ENV	Must be set to 'production'	Enables Secure cookie flag
DB_SSL	Must be 'true' for managed cloud databases	Required for Render/Supabase
CORS_ORIGIN	Exact production domain, never '*'	Prevents cross-origin attacks
Git History	Confirm .env never committed	git log --all -- backend/.env
HTTPS	TLS via Nginx or cloud load balancer	Never serve HTTP in production

APPENDIX C: ML ALGORITHMS QUICK REFERENCE

Algorithm	Type	Library	Key Parameters	Evaluation Metric	Result
DBSCAN	Spatial Clustering	scikit-learn	eps=300m, MinPts=4, haversine	Silhouette Score	0.74 (Strong)
KDE	Density Estimation	scikit-learn	bandwidth=500m, grid=30x30, Gaussian	Visual Inspection	Strong coverage
Prophet	Forecasting	prophet	yearly+weekly seasonality, 30 periods	80% CI Coverage	Validated
Random Forest	Classification	scikit-learn	n=200 trees, depth=15, balanced	Weighted F1	0.862 ± 0.018
Ridge Regression	Risk Scoring	scikit-learn	alpha=1.0, StandardScaler	R^2 Score	> 0.60
SHAP LinearExplainer	Explainability	shap	interventional perturbation	Qualitative	Deployed
Isolation Forest	Anomaly Detection	scikit-learn	contamination=0.05, random_state=42	Anomaly F1	Validated
Moran's I	Spatial Statistics	esda (PySAL)	k=5 neighbors, row-standardized W	p-value	< 0.001 all types
PAI	Evaluation Metric	Custom (haversine)	radius=300m, temporal holdout	PAI Value	> 100 (Excellent)
OR-Tools VRP	Route Optimization	ortools	PATH_CHEAPEST_ARC + GLS, 10s limit	Total distance	Optimal
Knox Test	Near-Repeat	Custom (haversine)	spatial=400m, temporal=14 days	Risk Score 0-10	Deployed
spaCy NER	NLP	spacy	en_core_web_sm, custom regex patterns	Entity Accuracy	Validated

Table C.1: ML Algorithms Quick Reference